Faculty of Engineering

Department of Informatics Engineering

Software Engineering and Information Systems

Integrated Social Media Platform (NOONIFY)

A Senior 1 project report - submitted to complete the requirements for obtaining a bachelor's degree in informatics engineering – Software Engineering and Information Systems

Prepared by

Sara Eyad Attah

Aya Al-salamat

Supervised by

Dr. Eng. Akram Massouh

May 2026

I Certify that the preparation of this project entitled

[Integrated Social Media Platform (NOONIFY)]

Prepared by

[Sara Eyad Atta - Aya Alsalamat]

was made under my supervision at Faculty of Informatics Engineering in partial Fulfillment of the Requirements for the Degree of Bachelor of Software Engineering and Information System

Name: .....

Signature: .....

Date: .....

Abstract

This project aims to design an integrated social media platform that allows users to create their accounts, share text and visual content, and interact with others in a safe and seamless way, the system relies on the MERN Stack (MongoDB, Express.js, React.js, Node.js) to cover the front and back sides, in addition to supporting tools such as Clerk for account management and user documentation, **Image Kit** for image processing and optimization, and **Ingest** for managing scheduled back-end tasks.

The platform offers a wide range of functions including Sign Up/Sign In, Profile Management, Posts, Stories, Social Interaction via Likes, Comments, and Unfollow, as well as a real-time chat system.

The platform also enables Search & Discover, real-time notifications, and media management using efficient cloud storage solutions.

Technically, the system is highly scalable thanks to the Node.js and MongoDB architecture, with an interactive and user-friendly user interface built with React.js.

Overall, the project offers a practical and integrated model of a modern social media app, combining a pleasant user experience with powerful performance powered by the latest Full Stack Development technologies.

Chapter One: Introduction

Project Importance

The importance of this project stems from the growing need for modern and secure social media platforms that enable users to interact and share content easily and efficiently. With the rapid advancement of web technologies and the increasing reliance on digital applications for daily communication, it has become essential to develop an integrated social networking system that combines high performance with an outstanding user experience.

This project contributes to enhancing developers' skills in the field of Full Stack Development through a practical implementation based on MERN Stack technologies (MongoDB, Express.js, React.js, Node.js). These technologies enable the development of a comprehensive system that covers all aspects: Front-End for designing an interactive user interface, Back-End for handling requests and managing data, and Database for storing information in an efficient and secure manner.

The importance of the project also lies in its integration of modern concepts and technologies such as Real-time Communication, media management using ImageKit, secure user authentication through Clerk, as well as background task management using Inngest.

Therefore, the project is not limited to being merely an educational experience; rather, it represents a comprehensive model that can be further developed into a real-world social media platform characterized by security, reliability, ease of use, and alignment with the latest technological advancements in the web domain.

Project Objectives

This project aims to develop a comprehensive social media platform that allows users to create accounts, manage their profiles, and share both textual and media content in a secure and seamless manner. It also seeks to provide an interactive environment that combines attractive design, high performance, and ease of use across various devices and platforms.

The primary objective is to build a web application based on the MERN Stack (MongoDB, Express.js, React.js, Node.js) to cover all system layers, from the Front-End interface to the Back-End system and the Database. The project also aims to enhance its technical aspects by integrating modern tools and services such as Clerk for user management and authentication, ImageKit for image processing and optimization, and Inngest for executing scheduled background tasks such as notifications and temporary data deletion.

Another important objective is the implementation of Real-time Features such as instant messaging (Real-time Chat) and real-time notifications (Real-time Notifications), in order to provide a dynamic and interactive user experience.

In addition, the project aims to build a Responsive User Interface (Responsive UI) using React.js and CSS, ensuring smooth navigation and user interaction.

In summary, the project aims to deliver a comprehensive practical model that bridges the gap between academic and practical aspects, enabling developers to apply their skills in building a professional Social Media Platform that highlights the core concepts of modern web technologies and best practices in full-stack development.

Related Studies for Social Media Platforms

First Study

Social Network Data Architecture and Storage

“TAO: Facebook's Distributed Data Store for the Social Graph”

This study discusses the TAO system developed by Facebook to efficiently manage relationships between users (Social Graph). The system aims to improve query speed and data access through a specialized, read-optimized data store, along with a unified API interface that simplifies querying without requiring an understanding of the underlying data structure. TAO is considered a model for designing databases tailored to large-scale social applications, as it combines MySQL with a caching layer to reduce response time and enhance performance.

Key Takeaways:

- Designing a specialized data layer tailored to the nature of social applications.
- Using caching and an API layer to reduce the load on databases.
- Geographical distribution of data improves access speed for users worldwide.

Source: Nathan Bronson et al., USENIX ATC, 2013 – Facebook Research.

Second Study

System Architecture and Scaling of Communication Platforms

"Scaling Big Data Mining Infrastructure: The Twitter Experience"

This study presents Twitter's experience in building an infrastructure capable of handling billions of messages daily while maintaining real-time performance. Twitter utilized technologies such as Kafka, HBase, and distributed storage systems to process and analyze continuous data streams efficiently.

The study emphasizes the importance of building a scalable system that allows component updates without service interruption, and achieving a balance between latency and throughput to provide a stable and interactive user experience.

Key Takeaways:

- Adopting streaming technologies and message queues such as Kafka is essential for real-time features.
- Building a flexible, scalable, and maintainable architecture without downtime.
- The importance of monitoring and logging tools in performance analysis.

Source: Twitter Engineering Reports, *Scaling Big Data Systems*.

Comparison Table Between Related Studies

Category	TAO - Facebook	Twitter	Our Project (NOONIFY)
Technologies Used	Data Store, Caching Layer, API	Kafka, HBase, Distributed Storage	MERN Stack
Advantages	High performance and fast data response	Real-time processing of large-scale data	Modern interfaces, efficient performance, integration of ready-made components
Disadvantages	Complex architecture and difficult maintenance	High operational cost and administrative complexity	Partial dependency on external services
Challenges	Maintaining consistency in distributed data	Balancing latency and availability	Ensuring security and privacy with high performance
Key Strengths	Specialized system for managing social relationships	Flexible architecture supporting real-time analytics	Integration of modern technologies with high flexibility and rapid development

Table 1: Comparison Between Related Studies

Gantt Chart

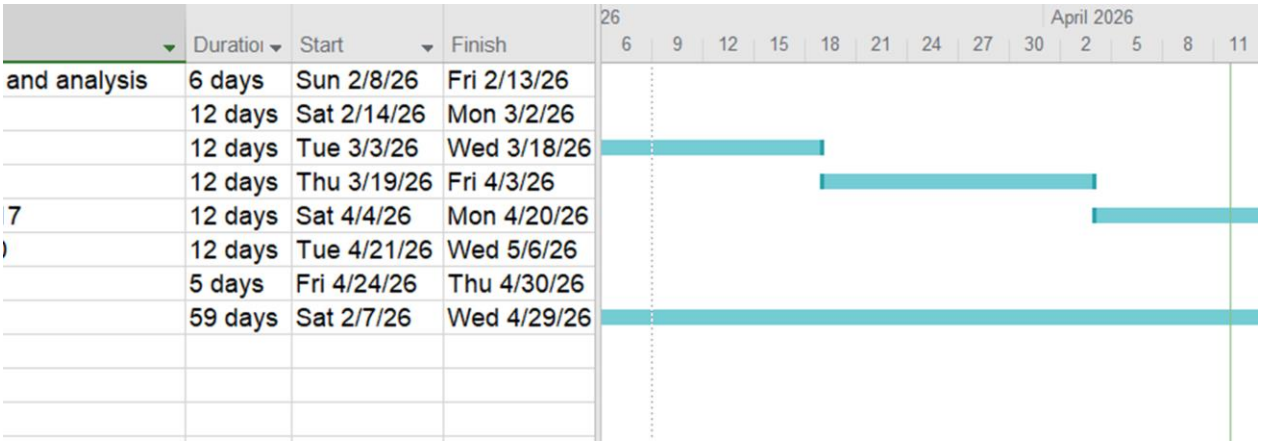


Figure 1: Gantt Chart

Project Development Methodology (SCRUM Methodology)

The SCRUM methodology was adopted as an Agile framework for implementing this project due to its high flexibility and its ability to organize the development process in an interactive and incremental manner (Iterative and Incremental Process). SCRUM is considered one of the most widely used Agile Software Development methodologies in modern software projects, as it focuses on dividing the work into short phases known as Sprints. During each Sprint, a set of high-priority tasks is implemented and continuously reviewed to improve the quality of the final product.

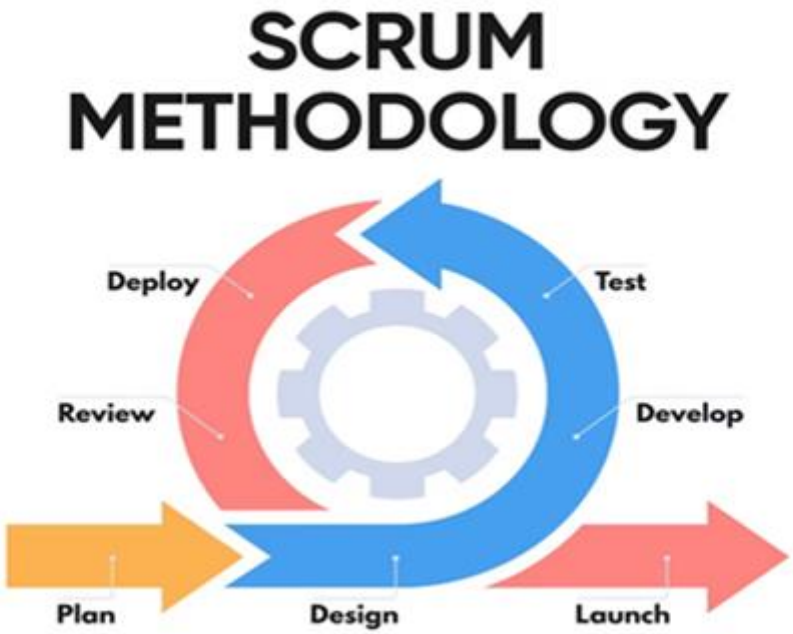


Figure 2: Scram Methodology Diagram

Applying SCRUM Methodology in the Social Media Platform Project

The development process was divided into multiple Sprints, where each Sprint focuses on a specific set of core functionalities within the platform.

The Sprints are as follows:

- **Sprint 1:** Setting up the development environment (Setup Environment) and implementing the Authentication System (Sign Up / Sign In).
- **Sprint 2:** Developing the Front-End Interface using React.js and integrating it with Back-End APIs.
- **Sprint 3:** Implementing search features and the friends system (Search & Friends System).
- **Sprint 4:** Developing the posts and comments module (Posts & Comments Module).
- **Sprint 5:** Adding the stories feature (Stories System) and notifications (Notifications).

After each Sprint, a **Sprint Review Meeting** is conducted to present progress and evaluate completed tasks, followed by a **Sprint Planning Meeting** to define the tasks for the next iteration.

System Architecture

First: Client–Server Architecture

The Client–Server architecture was adopted as the foundation for building the platform. It is one of the most widely used architectures in modern web applications, especially those based on communication between the user interface (Client) and the server (Server) via HTTP/HTTPS protocols.

In this project:

- The **Client** represents the user interface built using React.js.
- The **Server** represents the back-end developed using Node.js and Express.js, connected to the MongoDB database.

Data is exchanged between the client and server in JSON format through RESTful APIs.

Workflow

1. The user (**Client**) sends a request via the browser, such as logging in or viewing posts.
2. The **Server** receives the request through Express.js controllers and processes it based on the application logic.
3. The server communicates with the MongoDB database to retrieve or modify the required data.
4. The server sends a response back to the client, and the result is displayed through the interface using React.js.

Advantages of Client–Server Architecture

- Clear separation between the front-end and back-end (**Separation of Concerns**), which simplifies development and maintenance.

- Enhanced security, as core data processing occurs on the server rather than the client side.
- High scalability (**Scalability**), allowing independent scaling of servers or updates to the user interface.
- Reusability of back-end services through APIs that can be used in other applications (e.g., mobile apps).

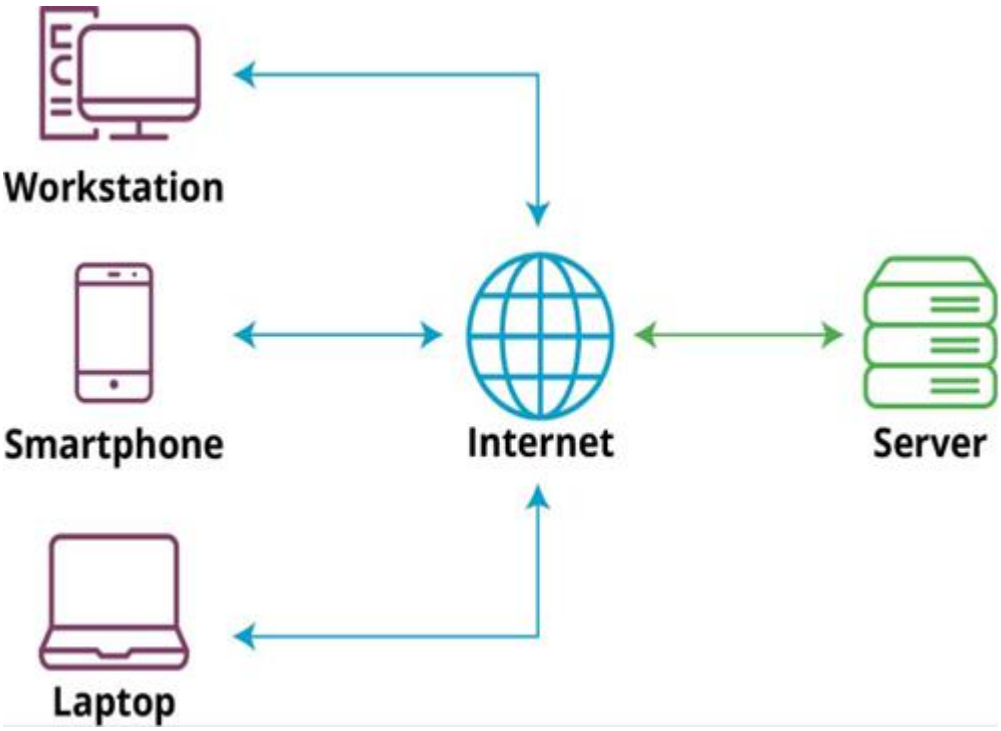


Figure 3: Client-Server

Second: MVC Architecture (Model – View – Controller)

In addition to the Client–Server architecture, the MVC pattern is implemented on the server side to organize the codebase and improve maintainability and scalability.

The MVC architecture divides the system into three main layers:

1. **Model:** Represents the data and its associated structures, including all operations related to data storage and retrieval from the database.
2. **View:** Represents the user interfaces displayed on the front-end using React.js.

3. **Controller:** Manages the interaction between the Model and the View. It handles user requests, executes the required business logic, and returns the results to the interface.

MVC Implementation in This Project

Controllers Used:

- **MessageControl:** Responsible for managing messages and conversations.
- **PostControl:** Responsible for creating, updating, and deleting posts.
- **StoryControl:** Responsible for uploading and managing stories.
- **UserControl:** Responsible for user registration and profile updates.

Models Used:

- **Email, Message, Post, Story, User, Connection**

These represent the collections in the MongoDB database and define the attributes and relationships between them.

View:

The user interfaces built using React.js and CSS, which receive data from the server through RESTful APIs and display it to users in an interactive and user-friendly manner.

Advantages of MVC Architecture

1. **Code Organization:** Dividing the code into separate components makes it easier to read and maintain.
2. **Facilitates Team Development:** Front-end and back-end teams can work in parallel without conflicts.
3. **Reusability:** Models can be reused in other services or future projects.
4. **Flexibility:** Any part of the system can be updated without affecting other components.

5. **Testability:** Each layer can be tested independently (Unit Testing).

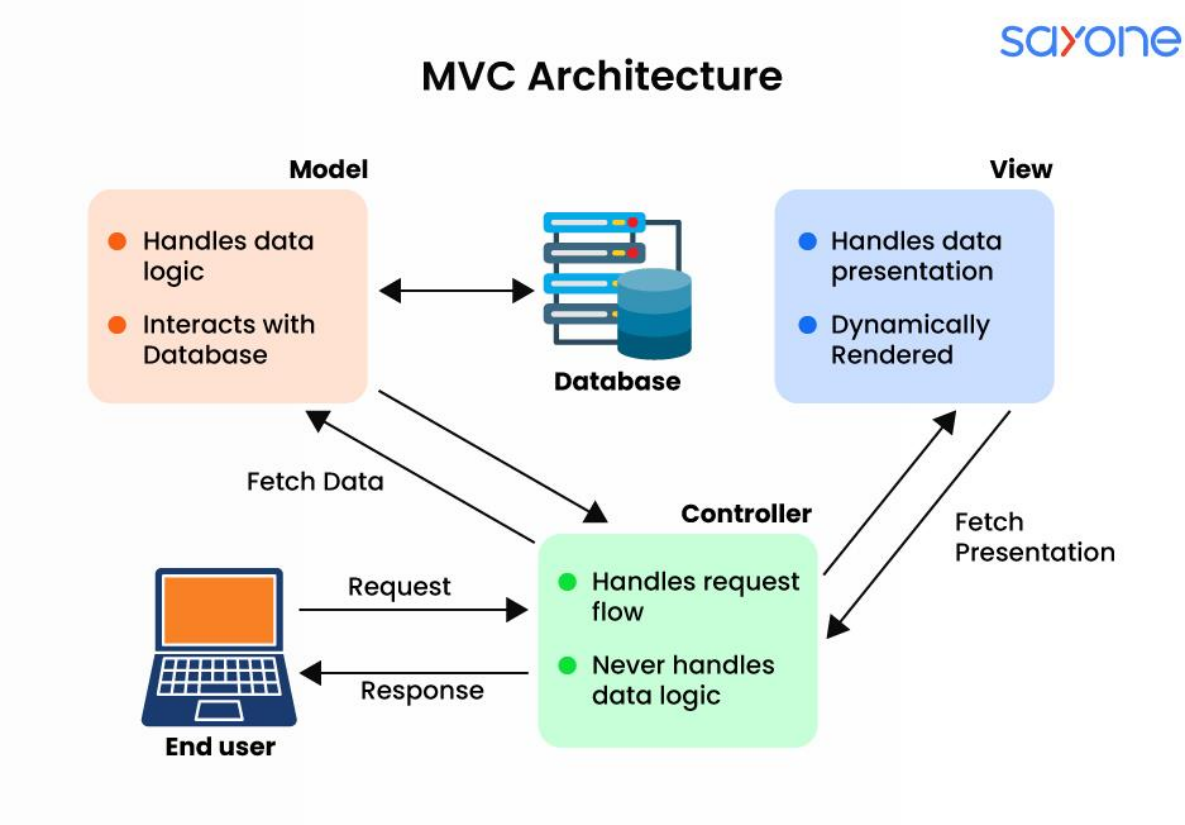


Figure 4: MVC

Third: Integration of Client–Server and MVC in the Project

The system in this project is considered a hybrid of both architectures:

Externally, the system is based on the **Client–Server Architecture**, where the user interacts with the application through the client interface, which communicates with the server over the internet. Internally, the server is structured according to the **MVC pattern**, which organizes the data flow within the system between the Model, Controller, and View layers. This combination makes the system scalable and easy to maintain.

System Layers

- **Controller Layer:**

- Handles incoming requests and executes the required logic through modules such as MessageControl, PostControl, StoryControl, and UserControl.

- **Model Layer:**

Manages the data represented by models such as User, Post, Story, Message, Connection, and Email, and interacts directly with the MongoDB database.

- **View Layer:**

Sends the processed data to the front-end, where it is presented to the user.

External Integrations

The system is also integrated with external services such as:

- **Clerk** for authentication and user management
- **ImageKit** for media handling and optimization
- **Inngest** for background task processing

Data Flow

The data flows through the system in a clear sequence:

User → Interface (Client) → Server (Controllers & Models) → Database → Back to Client (View)

Conclusion

This architecture achieves an optimal balance between **network-based distribution (Client–Server Architecture)** and **internal system organization (MVC Pattern)**. As a result, the platform becomes robust, secure, scalable, and easier to maintain and extend in future development stages.

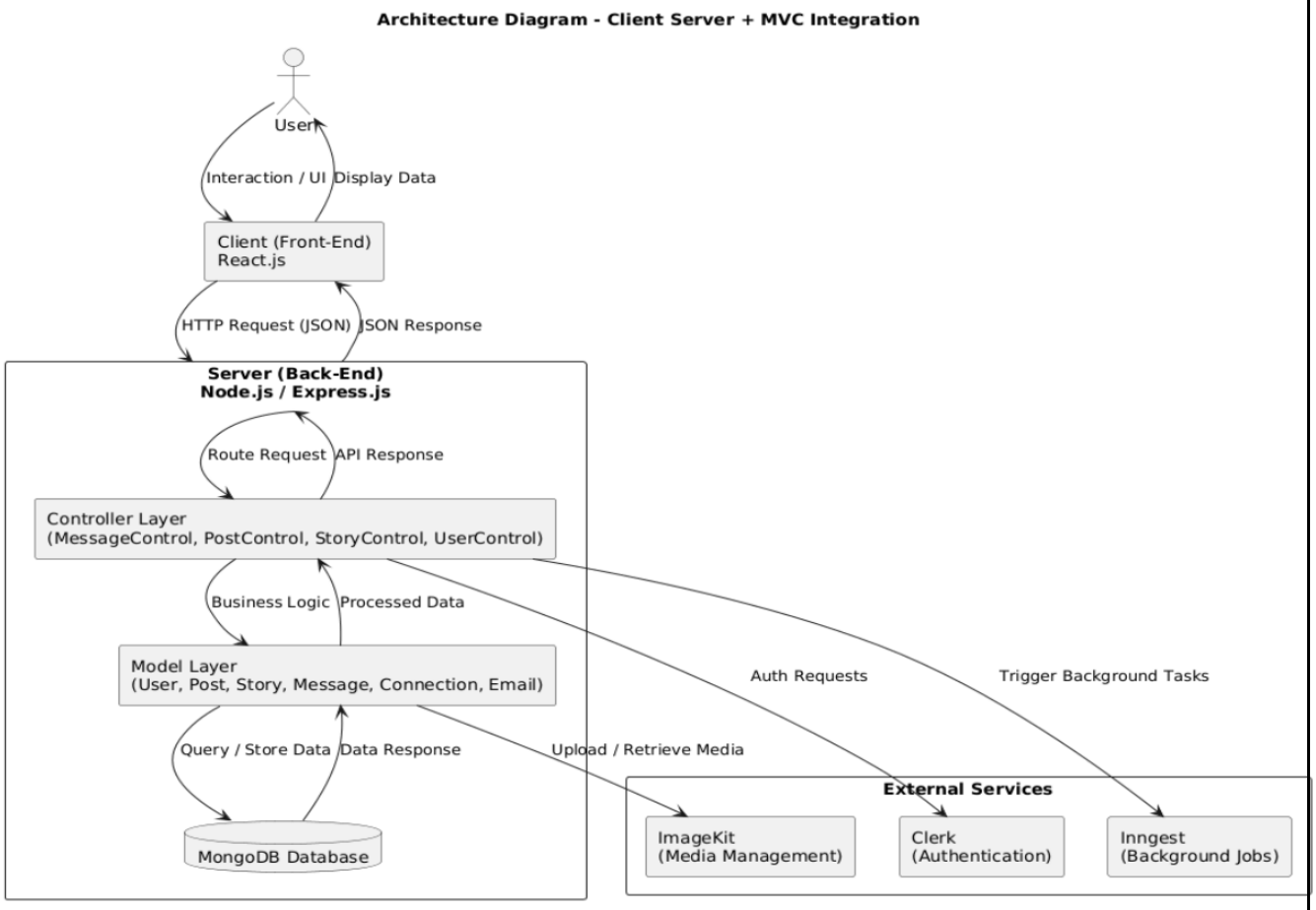


Figure 4: Integration of the Two Architectures

Tools Used

This chapter presents the tools and technologies utilized in developing the Noonify social media platform. The tools are categorized into three main groups: Front-End tools, Back-End tools, and Database tools, in alignment with the MERN Stack architecture.

Front-End Tools

The front-end of the application was developed using a set of modern tools aimed at building a responsive and interactive user interface :

- **React.js**
- A JavaScript library used for building component-based user interfaces, which facilitates code reusability and improves maintainability.
- **Vite**

A modern build tool used to accelerate the development process and run the project more efficiently compared to traditional tools.

- **JavaScript (ES6+)**

The primary programming language used to implement front-end logic, handle events, and manage user interactions.

- **HTML5**

Used to structure web pages.

- **CSS3**

Used to style user interfaces and enhance the visual experience.

- **Axios**

A library used to send and receive HTTP requests between the front-end and back-end.

- **ESLint**

A tool used for code analysis, detecting errors, and improving code quality.

- **Vercel**

A platform used to deploy and host the front-end application in a production environment.

Back-End Tools

The back-end was developed using technologies that ensure high performance and structured system logic. Key tools include:

- **Node.js**

A JavaScript runtime environment used to execute server-side code efficiently.

- **Express.js**

A framework built on Node.js used to create RESTful APIs and manage routing.

- **JavaScript**

Used as the primary language for implementing server-side logic.

- **JWT (JSON Web Token)**

Used to implement authentication and verify user identity.

- **Multer**

A library used for handling file uploads (such as images) from users to the server.

- **NodeMailer**

A library used for sending emails, such as verification and notification messages.

- **ImageKit**

A service used for managing, uploading, and storing images in the cloud.

- **Middleware**

Used for handling intermediate requests such as authorization and route protection.

Database Tools

A flexible and scalable database solution was used to store user data and content. The tools include:

- **MongoDB**

A NoSQL database based on documents, known for its flexibility and ease of handling unstructured data.

- **Mongoose**

An Object Data Modeling (ODM) library for MongoDB that simplifies schema creation and data operations.

Summary of Tools Used

The project adopted the MERN Stack as its primary architecture, achieving integration between the front-end, back-end, and database. This integration enabled the development of a modern, well-structured, and scalable social media platform that meets both academic and practical project requirements.

Chapter Two: Analytical Study

General Requirements

Authentication System

- Ability to create a new account (Sign Up) using email or phone number.
- User login (Sign In) using credentials or via social login (Google / GitHub).
- Password reset functionality (Reset Password) in case it is forgotten.
- Logout from the current session (Log Out).

User Profile Management

- Display basic user information (name, profile picture, bio, number of followers).
- Edit profile (Edit Profile) and update user data.
- View other users' profiles (View Other Profiles).
- Display online/offline status (Online / Offline Status).

- Secure authentication and session management using Tokens and Clerk Authentication.

Posts Management

- Create new posts (Create Post) with text or images.
- Edit or delete posts (Edit / Delete Post).
- Display posts on the home page (Feed) in chronological order.
- Interact with posts through likes (Like) and comments (Comment).
- Save favorite posts (Save Post) or share them (Share Post).
- Report inappropriate posts (Report Post).

Stories / Status System

- Upload a new story (Upload Story) as an image or short video.
- View stories from followed users (View Stories).
- Delete stories manually or automatically after 24 hours.
- Display viewers list (Viewers List).

Real-time Chat & Messages

- Start a new conversation (Start Conversation) between users.
- Send and receive text messages, images, and emojis.
- Support real-time chat using Socket.io.
- Display message status (Sent – Delivered – Read).
- Mute or delete conversations (Mute / Delete Chat).

Connections System

- Follow or unfollow users (Follow / Unfollow).
- Send and receive friend requests (Friend Requests).
- Display followers and following lists (Followers / Following Lists).
- Suggest friends based on shared interests (Friend Suggestions).
- Block or unblock users (Block / Unblock).

Search & Discover System

- Search for users by name or username (Username Search).
- Search for users based on location.
- Search for posts using hashtags (Hashtags).
- Display a Discover page containing trending posts and suggested users.

Notifications System

- Send real-time notifications when interacting with posts or receiving new follows.
- Display all notifications on a dedicated page (Notifications Page).
- Ability to delete notifications or mark them as read (Mark as Read).

Media Management

- Upload images using ImageKit with automatic optimization.
- Support multiple image formats (JPEG, PNG, WEBP).
- Compress images to reduce size without losing quality.
- Protect media URLs from unauthorized access (Access Control).

Backend Operations

- Provide APIs for interacting with users, posts, and messages.
- Execute scheduled tasks using Inngest (e.g., deleting expired stories).
- Perform error logging and activity monitoring (Error Logging & Monitoring).

Non-Functional Requirements

- **Security:** Use Clerk for authentication and HTTPS for data encryption.
- **Performance:** High loading speed, real-time responsiveness, caching, and real-time chat support.
- **Scalability:** Support horizontal scaling using Node.js and MongoDB.
- **Usability:** Interactive interface with responsive design (Responsive UI).
- **Reliability:** Stable operation with system monitoring (Logging & Monitoring).
- **Maintainability:** Well-structured modular code with proper documentation.

- **Compatibility:** Support for all browsers and devices (Cross-Platform).
- **Privacy:** Protection of user data with flexible privacy settings.

Requirements to Be Implemented in the Project

Requirement ID	Functional Requirement	Priority
ID-01	Start a new conversation between users	1
ID-02	Send and receive text messages and emojis	1
ID-03	Support real-time messaging	1
ID-04	Display message status (Sent – Delivered – Read)	1
ID-05	Delete or mute conversations	1
ID-06	Follow or unfollow users	2
ID-07	Block or unblock users	2
ID-08	Display online status (Online / Offline)	2
ID-09	Search for posts using hashtags	2
ID-10	View the Explore page containing highly interactive posts and suggested users	2
ID-11	Send real-time notifications when interacting with posts or when new followers appear	3
ID-12	View all notifications on a dedicated page	3
ID-13	Delete notifications or mark them as read	3
ID-14	Upload images and automatically optimize them with support for multiple formats	4
ID-15	Protect media links from unauthorized access	4
ID-16	Log errors and system activities	4
ID-17	Execute scheduled tasks such as deleting expired stories	4
ID-18	Report inappropriate or violating posts	4
ID-19	Suggest friends or groups based on shared interests	5
ID-20	Discover trending content in real time	5

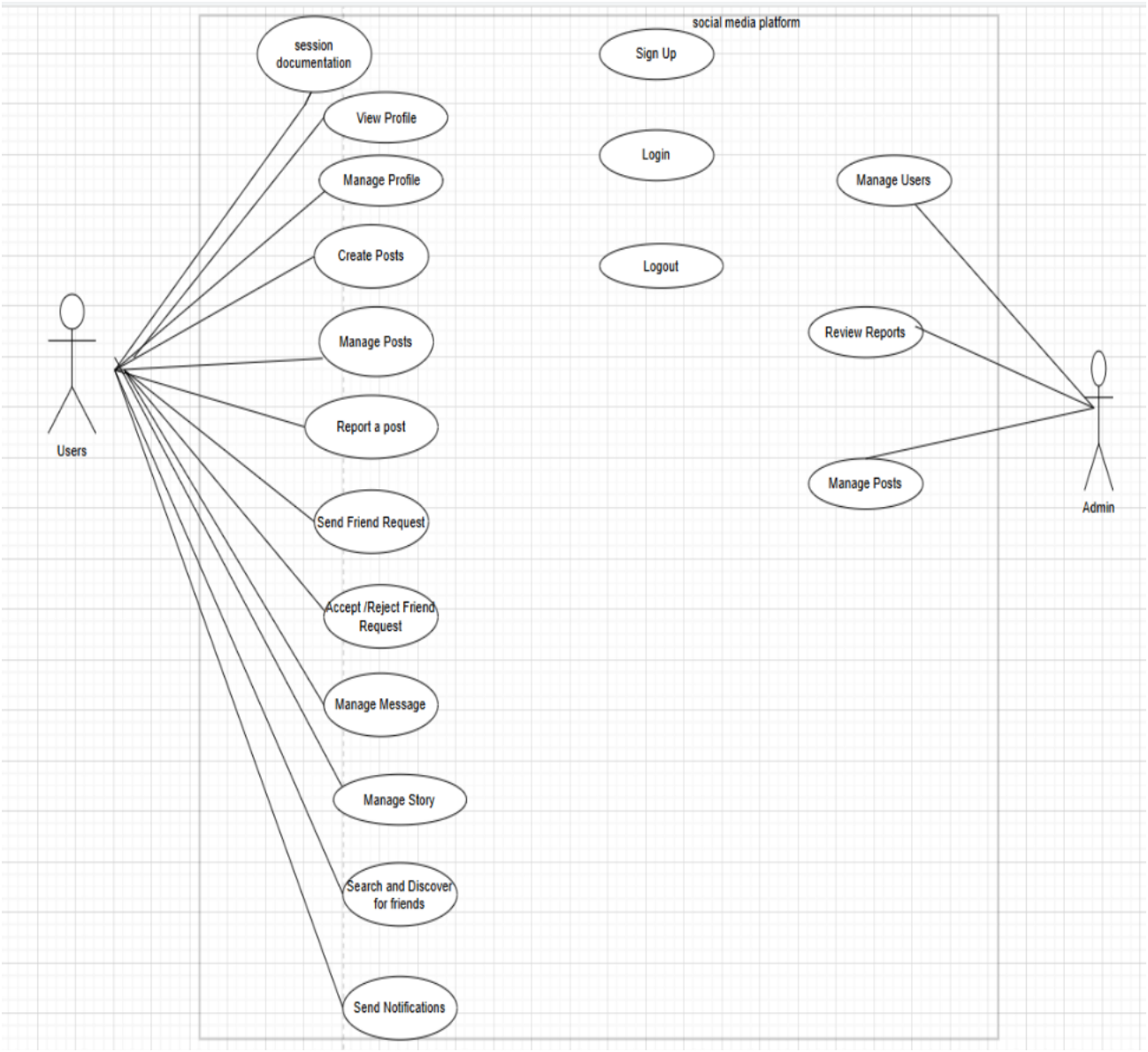


Figure 5: Use Case Diagram

Use Case Specifications

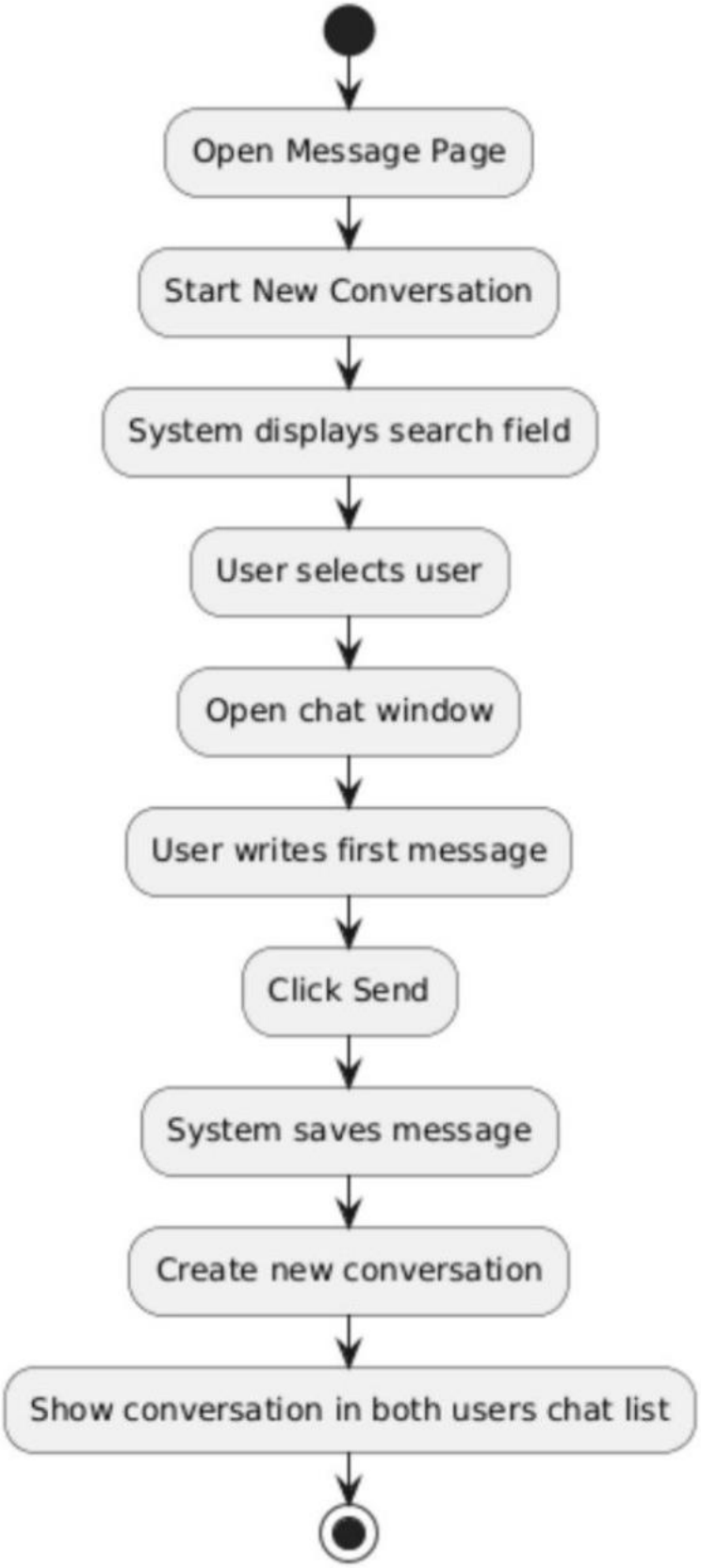
Activity Diagrams

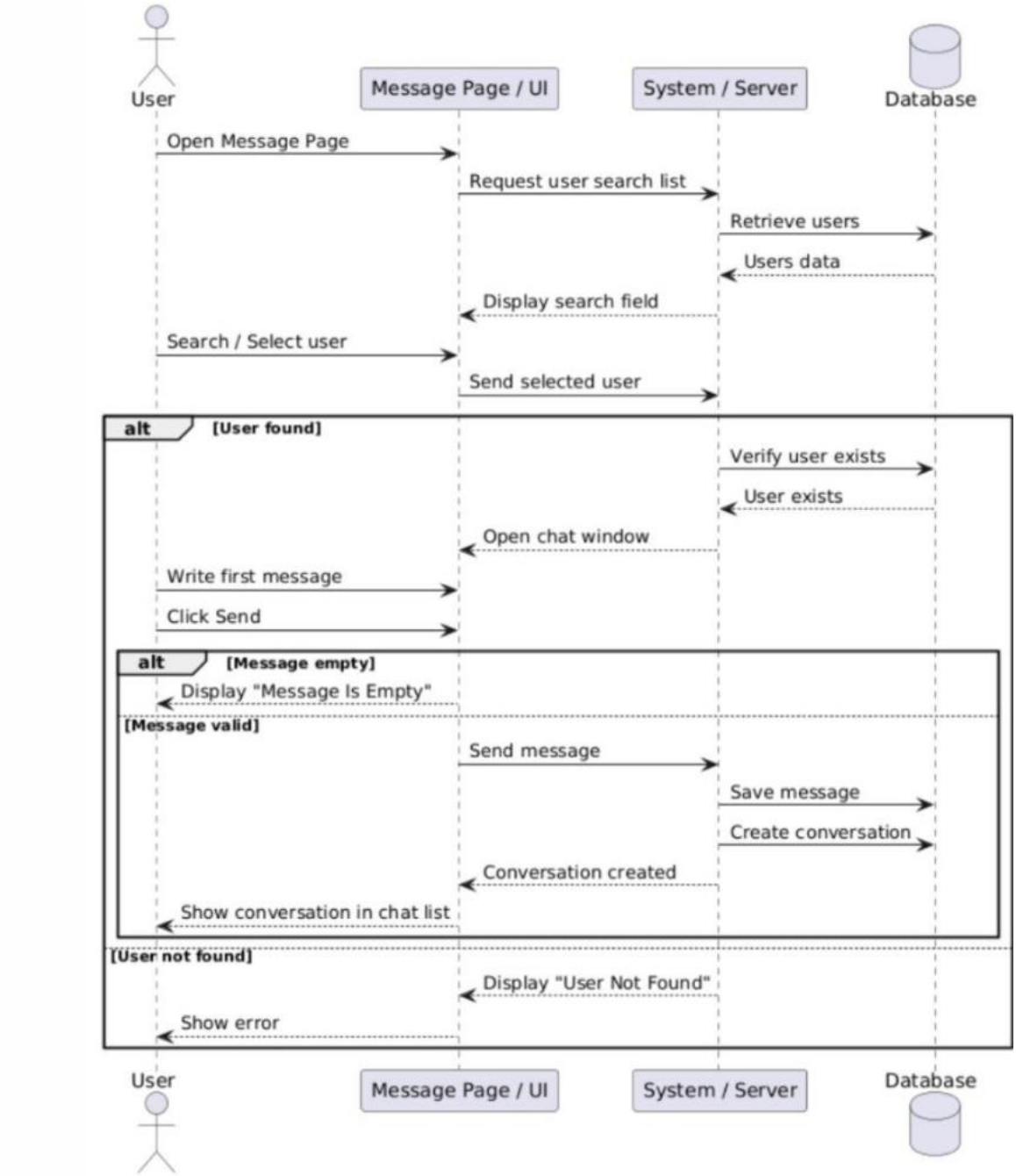
Sequence Diagrams

Use Case Name/ID	Start New Conversation UC_19

Actors	User
Description	This case allow the user to start a new conversation with another user by selecting the user and sending the first message.
Precondition	1. The user must be logged into the system. 2. The selected user must be exist in the system.
Main flow	1.The user open the Message Page and start new conversation. 2.The system displays a list of search field to find user. 3.The user selects the user. 4.The system opens a chat window. 5.The user writes the first message and click send. 6.The system saves the message and creates a new conversation and it's appear in both user's chat list.
Alternative flow	A3: The system cannot find the user and displays "User Not Found". A5:The user clicks Send without writing a message and display "Message Is Empty".
Postconditions	A new conversation is created in the system and the conversation appears in the user's chat list.[iii]

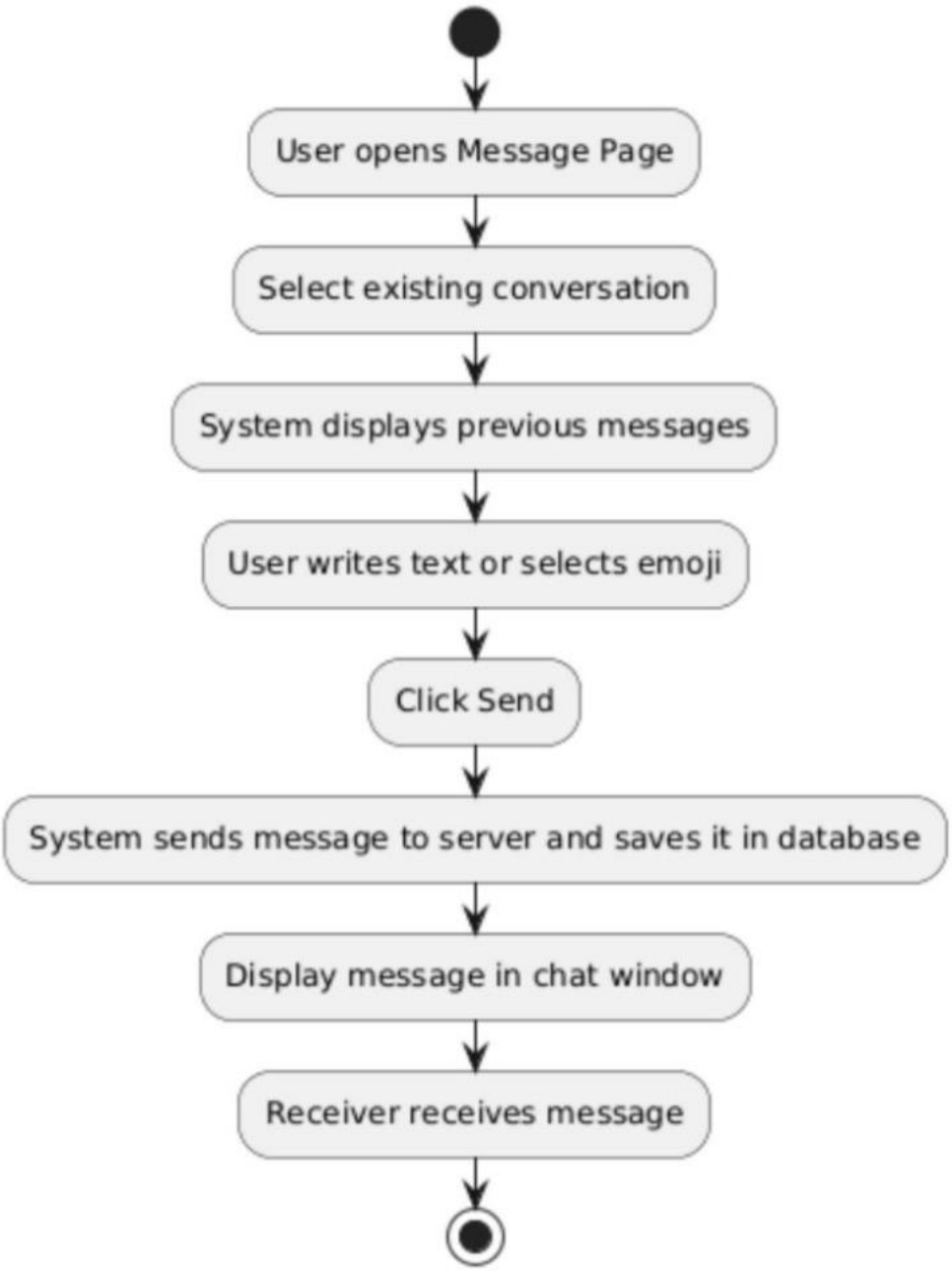
Table 2: Use Case Specifications Start New Converation

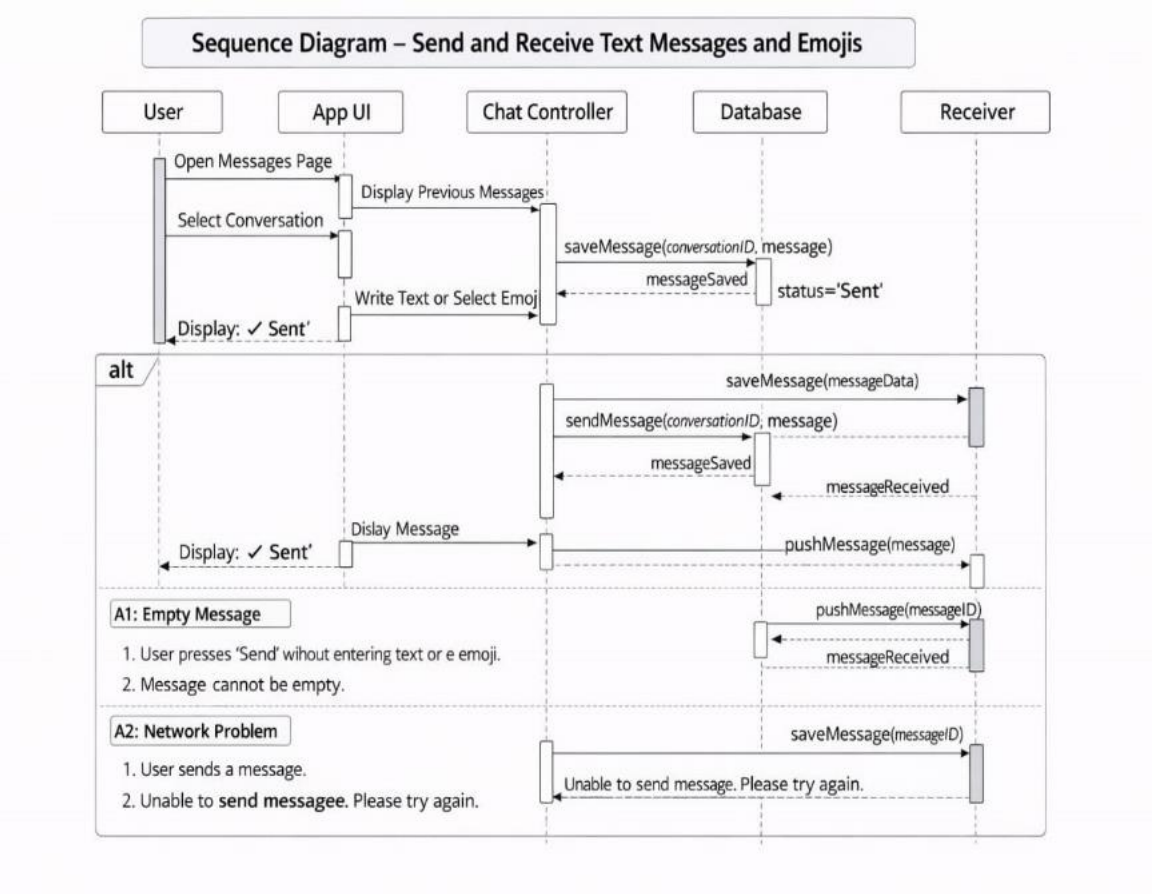

Open Message Page
Start New Conversation
System displays search field
User selects user
Open chat window
User writes first message
Click Send
System saves message
Create new conversation
Show conversation in both users chat list



Use Case Name/ID	Send and Receive Text Message and Emojis UC_20
Actors	User

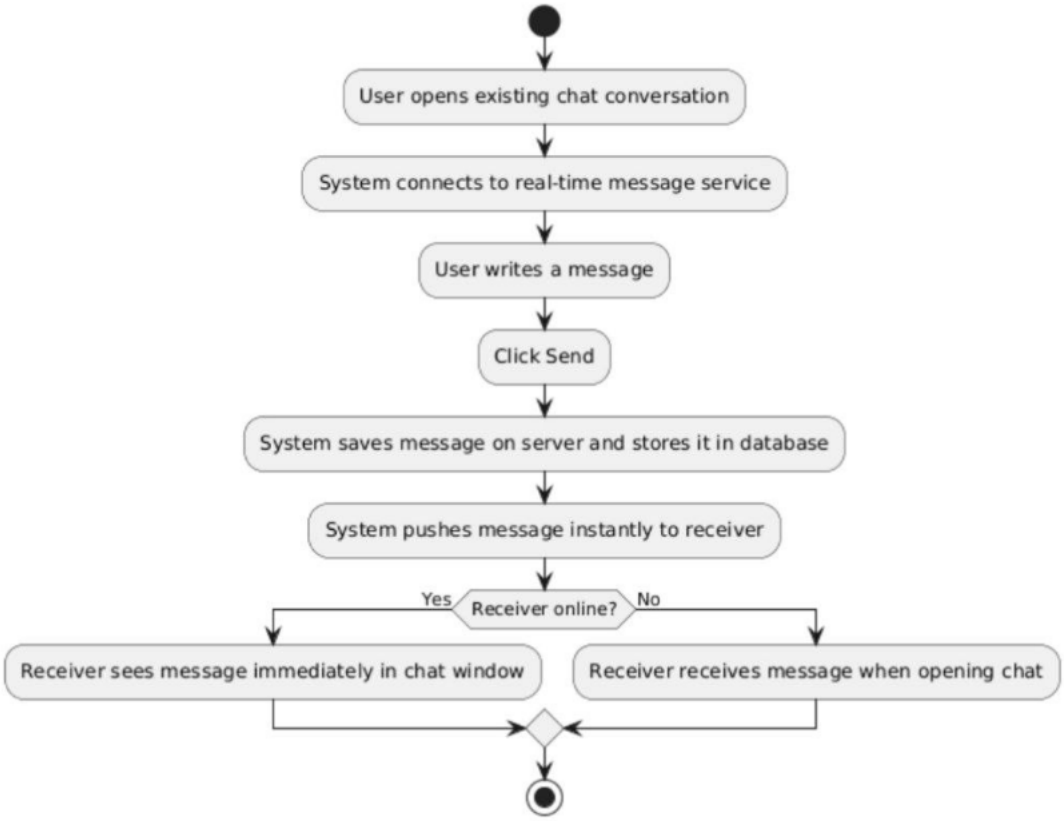
Description	This case allow the user to start send and receive text message and emojis within an existing conversation in the chat system.
Precondition	1.The user must be logged into the system. 2.A conversation between users must already exist.
Main flow	1.The user open the Message Page and select an existing conversation. 2.The system displays previous message. 3.The user writes a text message or selects an emoji and click Send. 4.The system sends the message to the server and saves in the database and the message displays in the chat window. 5.The receiver receives the message.
Alternative flow	A3:The user clicks Send without writing a message and display ”Message Is Empty”.
Postconditions	The message appears in the conversation window.

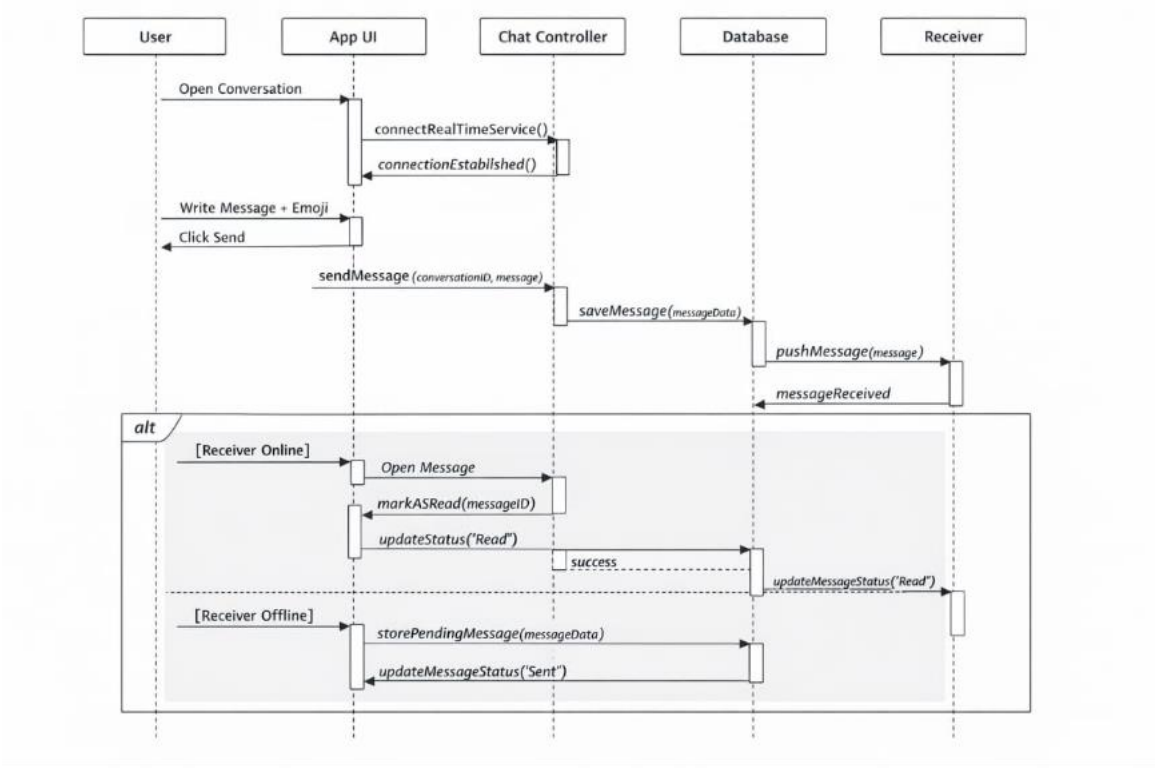

User opens Message Page
Select existing conversation
System displays previous messages
User writes text or selects emoji
Click Send
System sends message to server and saves it in database
Display message in chat window
Receiver receives message



Use Case Name/ID	Support Real-Time Message UC_21

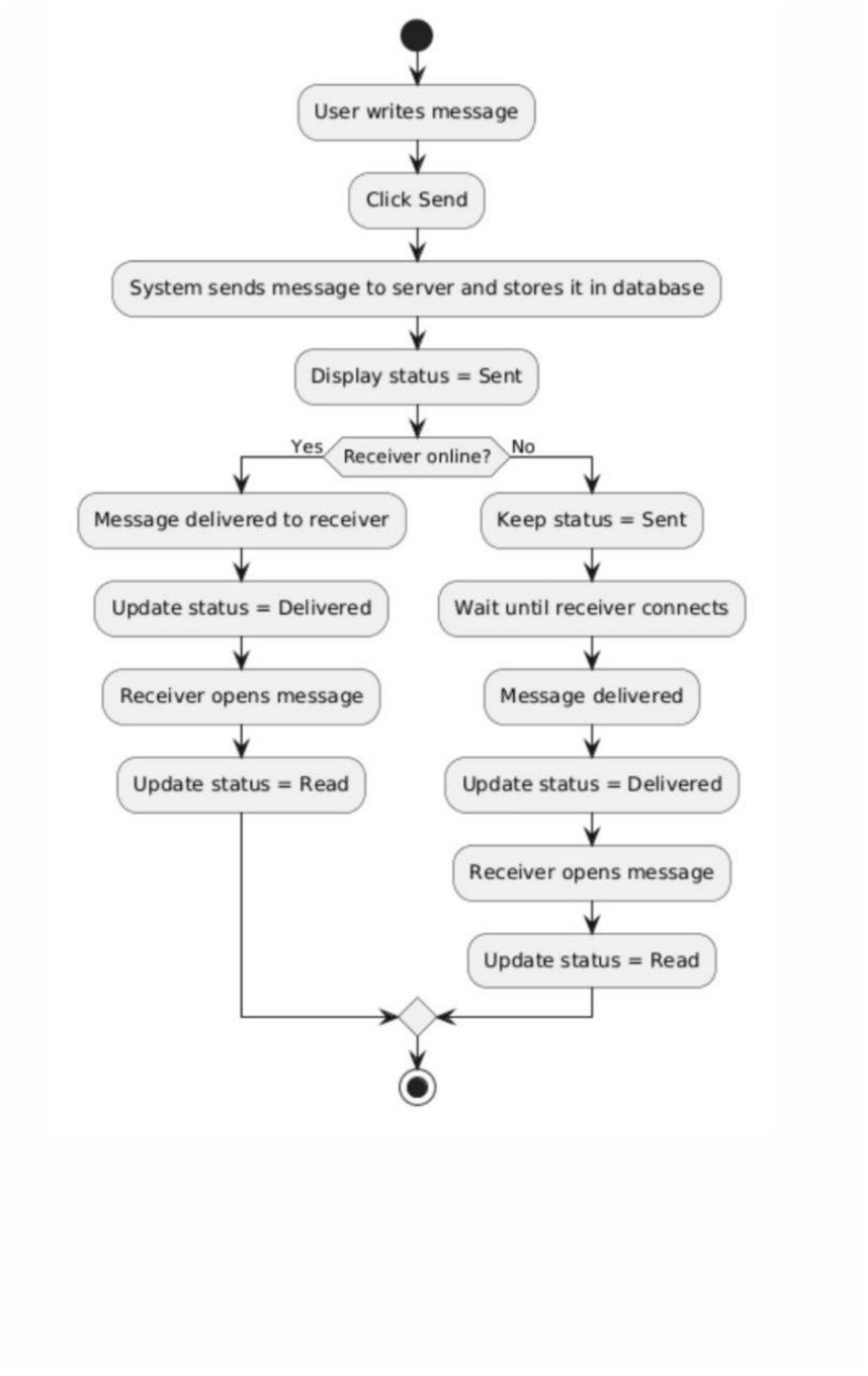
Actors	User
Description	This case allow the user to start send and receive text message instantly in real time within an active conversation without refreshing the page.
Precondition	1. The user must be logged into the system. 2. The user must open an existing conservation.
Main flow	1.The user open the chat conversation. 2.The system connect to the real time message service. 3.The user writes a message and click send. 4.The system saves the message to the server that stores it in database and pushed the message instantly to the receiver. 5.The receiver sees the message immediately in the chat window.
Alternative flow	A5: The receiver is offline it receives the message when they open the chat.
Postconditions	The message appears immediately in chat window.

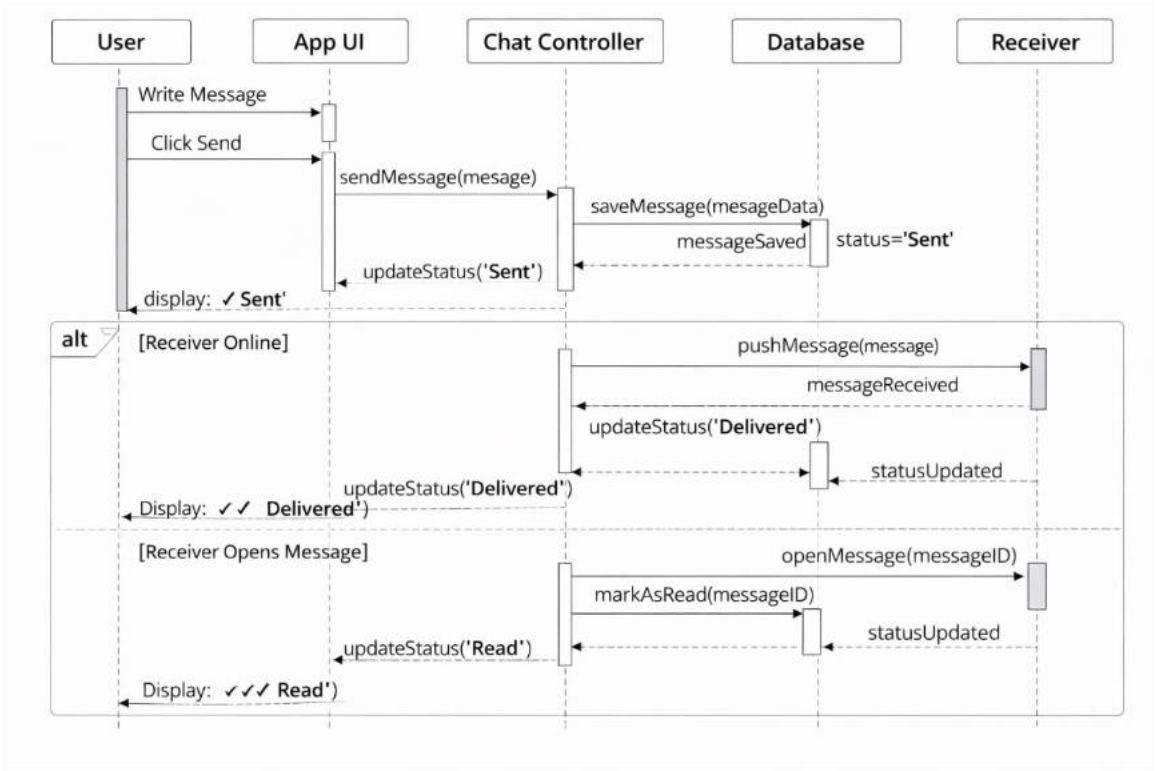

User opens existing chat conversation
System connects to real-time message service
User writes a message
Click Send
System saves message on server and stores it in database
System pushes message instantly to receiver
Receiver online?
Yes
No
Receiver sees message immediately in chat window
Receiver receives message when opening chat



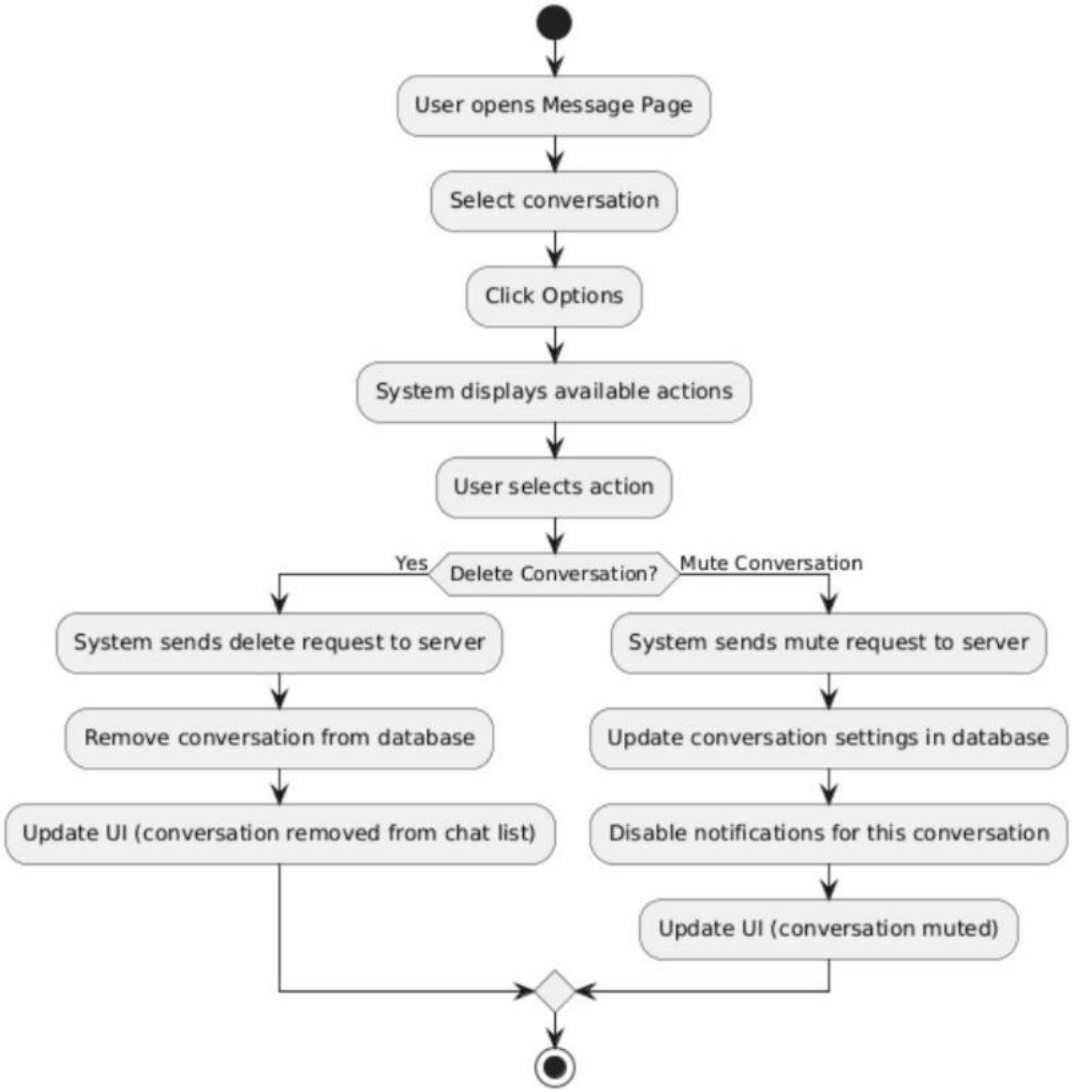
Use Case Name/ID	Display Message Status (Sent –Delivered – Read) UC_22

Actors	User
Description	This case allow the system to display the status of a message in a conversation, indicating whether the message is sent ,delivered ,read.
Precondition	1. The user must be logged into the system. 2. The user must send a message.
Main flow	1. The user writes a message and click send. 2.The system send the message to the server that stores it in database and displays Sent status. 3.When the receiver receives the message, the system updates the status to Delivered. 4. When the receiver opens the message, the system updates the status to Read.
Alternative flow	A3: The receiver is offline it receives the system displays Sent until the receives connects when its connect ,the status change to Delivered.
Postconditions	The sender can see the message status and the receiver's action update status automatically.


User writes message
Click Send
System sends message to server and stores it in database
Display status = Sent
Receiver online?
Yes
No
Message delivered to receiver
Update status = Delivered
Receiver opens message
Update status = Read
Keep status = Sent
Wait until receiver connects
Message delivered
Update status = Delivered
Receiver opens message
Update status = Read

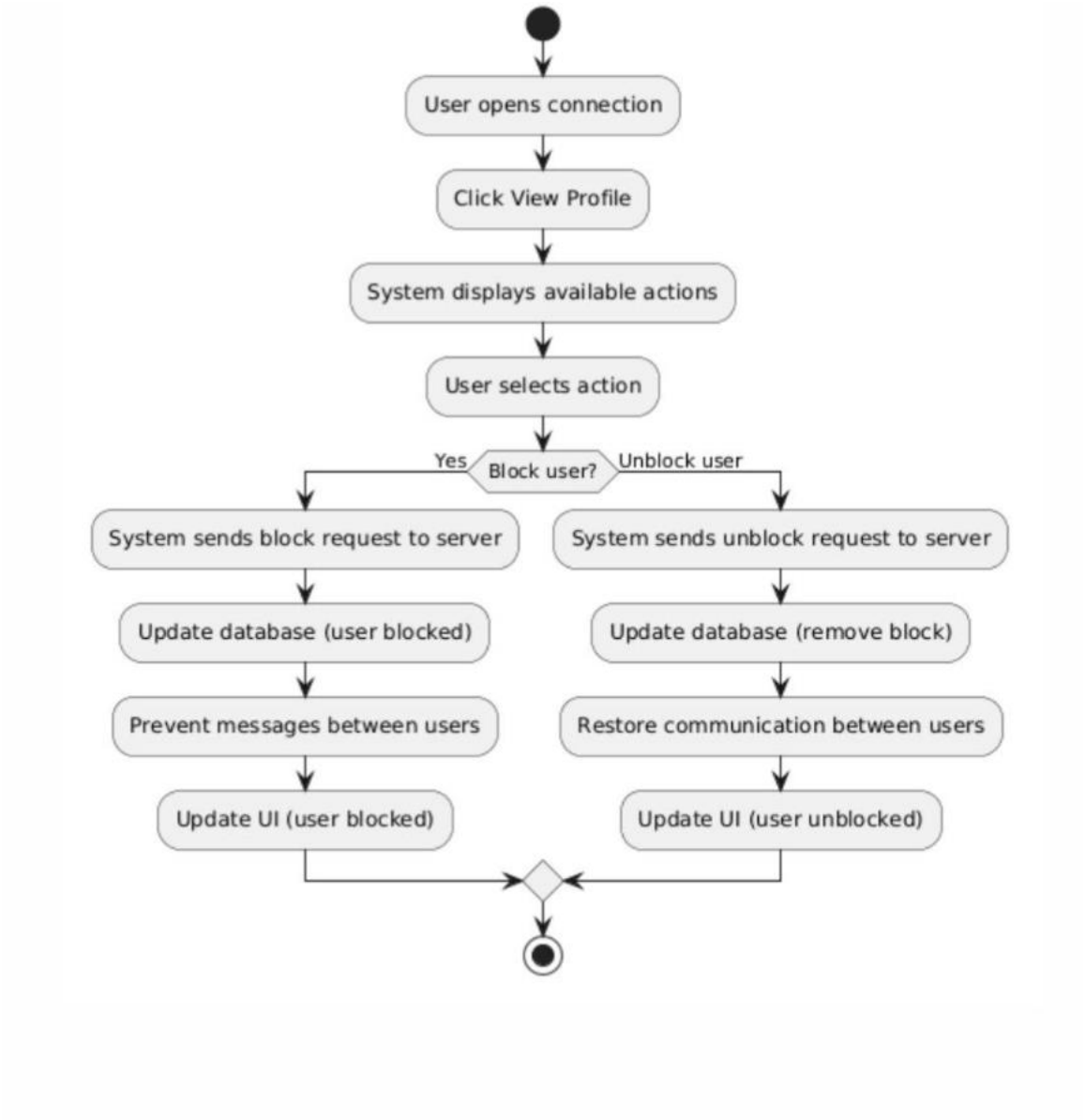

User
App UI
Chat Controller
Database
Receiver
Write Message
Click Send
sendMessage(mesage)
saveMessage(mesageData)
messageSaved
status='Sent'
updateStatus('Sent')
display: ✓ Sent'
alt
[Receiver Online]
pushMessage(message)
messageReceived
updateStatus('Delivered')
statusUpdated
updateStatus('Delivered')
Display: ✓✓ Delivered')
[Receiver Opens Message]
openMessage(messageID)
markAsRead(messageID)
statusUpdated
updateStatus('Read')
Display: ✓✓✓ Read')

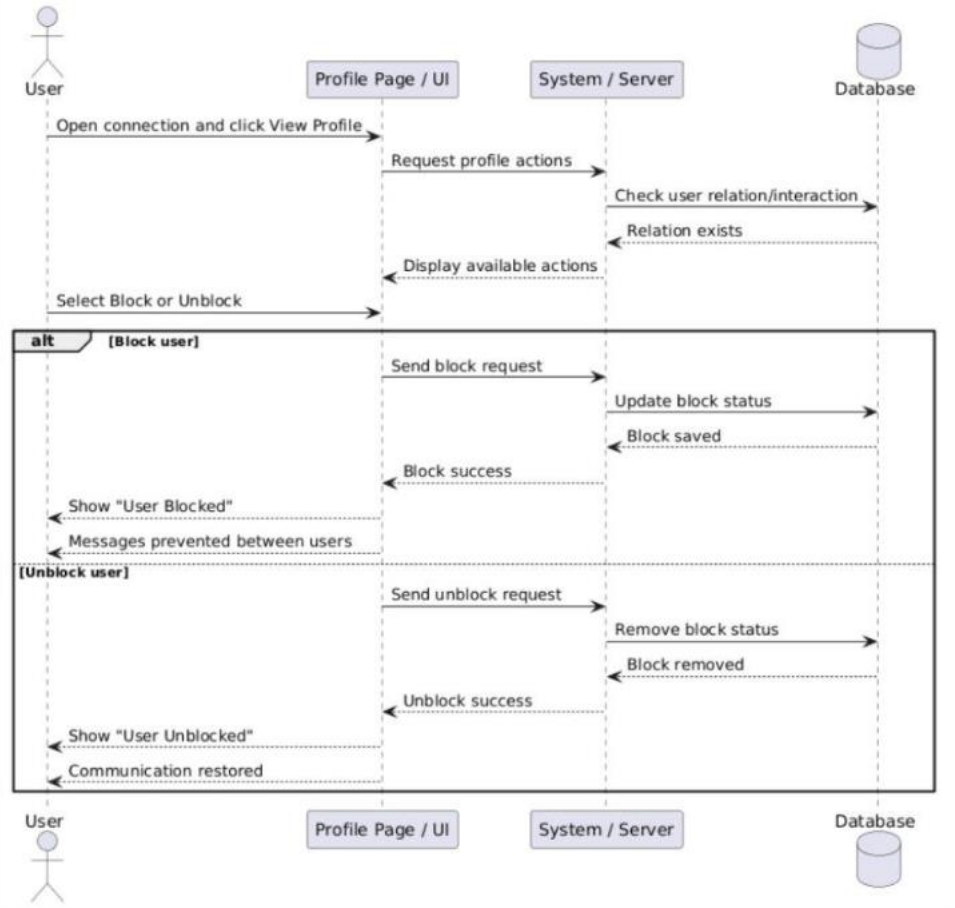
Use Case Name/ID	Delete or Mute Conversation UC_23
Actors	User
Description	This case allow the system to delete a conversation or mute notifications from a specific conversation.
Precondition	1. The user must be logged into the system. 2. The conversation must already exist.
Main flow	1. The user opens a Message Page, select a conversation and clicks Options. 2. The system displays available actions . 3. The user chooses Delete Conversation or Mute Conversation. 4. The system process the request , updates the database and update the UI .
Alternative flow	A3: if the user selects a delete conversation the system sends the request to the server , the system removes the conversation from the database ,the conversation disappears from the chat list. A3: if the user selects Mute Conversation, system sends a mute request to the serer, updates the conversation settings in the database ,notifications for that conversation are disabled.
Postconditions	The conversation is deleted from the user's chat list or notification for the conversation is muted.


User opens Message Page
Select conversation
Click Options
System displays available actions
User selects action
Delete Conversation?
Yes
Mute Conversation
System sends delete request to server
Remove conversation from database
Update UI (conversation removed from chat list)
System sends mute request to server
Update conversation settings in database
Disable notifications for this conversation
Update UI (conversation muted)

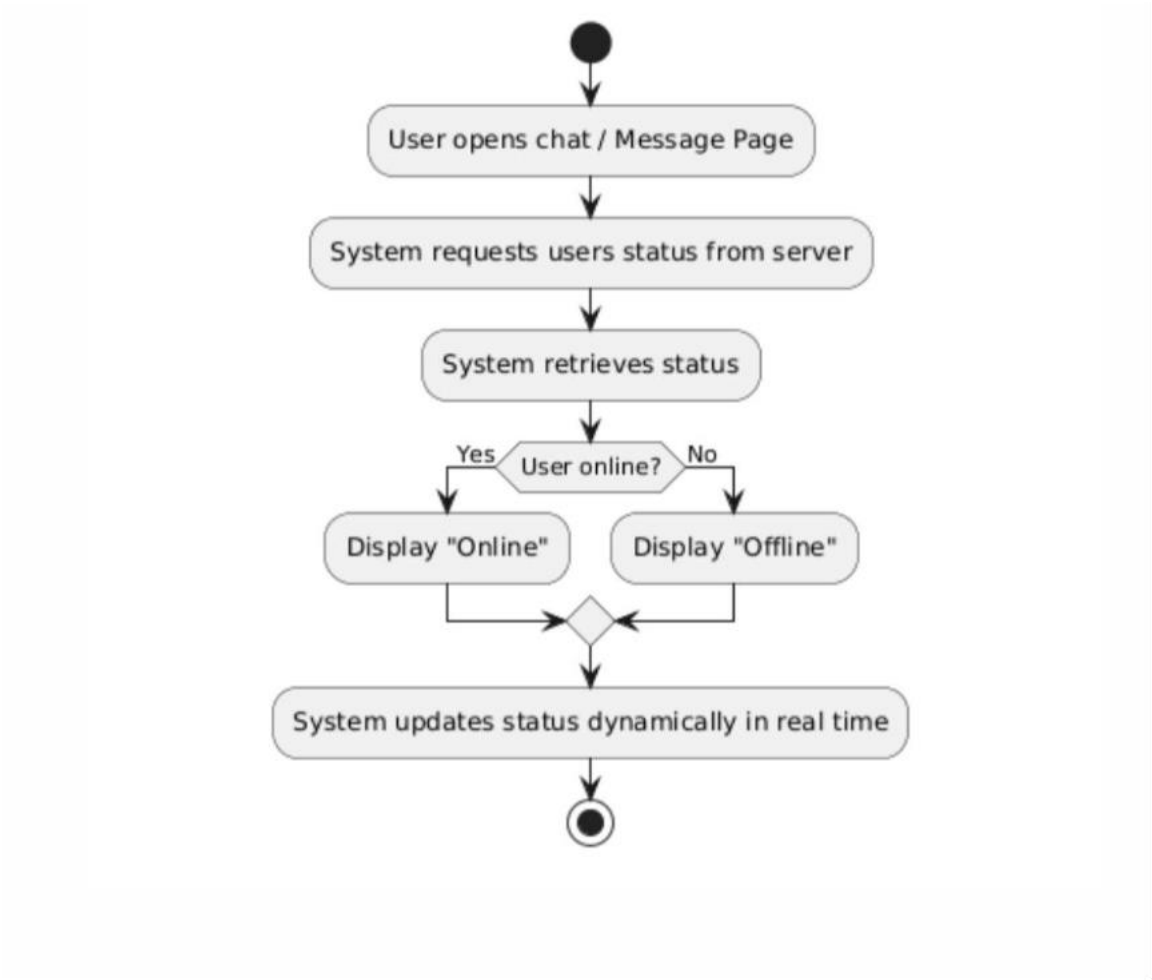

User
App UI
Chat Controller
DB
Open Messages Page
Select Conversation
Open Options
requestConversationOptions(conversationID)
displayOptions()
alt
[Delete Conversation]
Click Delete
deleteConversation(conversationID)
removeConversation(conversationID)
deleteSuccess
updateChatList()
[Mute Conversation]
Click Mute
muteConversation(conversationID)
updateMuteStatus(conversationID)
statusUpdated
updateConversationSettings()

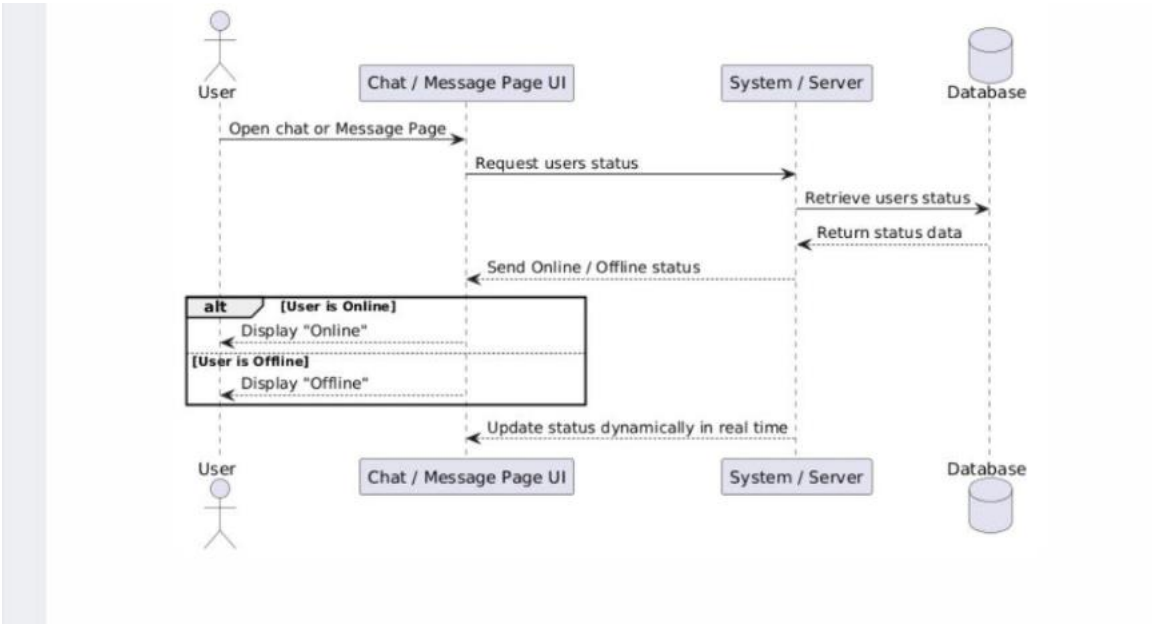
Use Case Name/ID	Block or Unblock User UC_24
Actors	User
Description	This case allow the user to block or unblock another user.
Precondition	1. The user must be logged into the system. 2. A relation or interaction between users exists.
Main flow	1. The user opens a Connection and click view Profile. 2. The system displays available actions. 3. The user selects Block or Unblock . 4. The system processes the requests , update the database, and updates UI.
Alternative flow	A3: if the user selects block the system blocks the user , Message prevented between the users. A3: if the user selects unblock the system removes the block , Communication is restored.
Postconditions	The blocked user cannot send message , if unblocked communication is restored.


User opens connection
Click View Profile
System displays available actions
User selects action
Block user?
Yes
Unblock user
System sends block request to server
System sends unblock request to server
Update database (user blocked)
Update database (remove block)
Prevent messages between users
Restore communication between users
Update UI (user blocked)
Update UI (user unblocked)

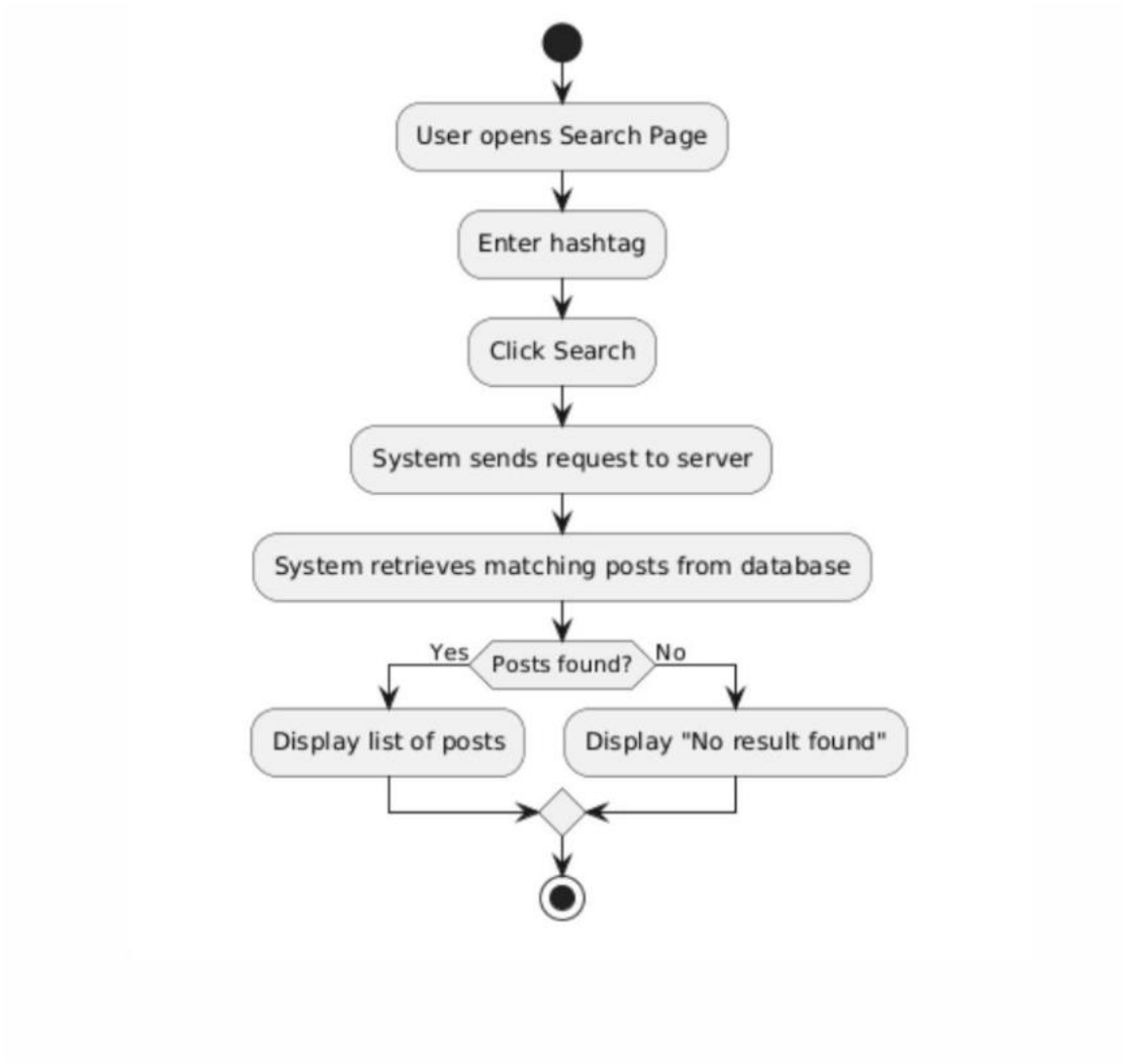

User
Profile Page / UI
System / Server
Database
Open connection and click View Profile
Request profile actions
Check user relation/interaction
Relation exists
Display available actions
Select Block or Unblock
alt
[Block user]
Send block request
Update block status
Block saved
Block success
Show "User Blocked"
Messages prevented between users
[Unblock user]
Send unblock request
Remove block status
Block removed
Unblock success
Show "User Unblocked"
Communication restored
User
Profile Page / UI
System / Server
Database

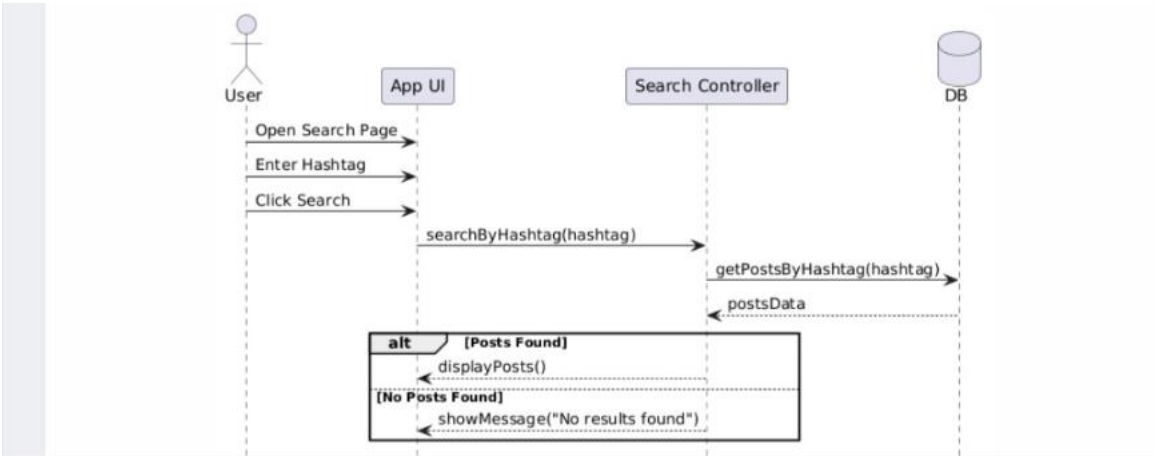
Use Case Name/ID	Display online |Status (Online / Offline) UC_25
Actors	User
Description	This case allow the user to display whether a user is online or offline in real time.
Precondition	1. The user must be logged into the system.
Main flow	1. The user opens the chat or Mesasage . 2. The system requests the status of other users , retrieves status from the server , display Online or Offline .
Alternative flow	A2: User Online : the system displays Online A2: User Offline : the system displays |Offline .
Postconditions	The user status is displayed correctly , and the system updates the status dynamically.


User opens chat / Message Page
System requests users status from server
System retrieves status
Yes
User online?
No
Display "Online"
Display "Offline"
System updates status dynamically in real time

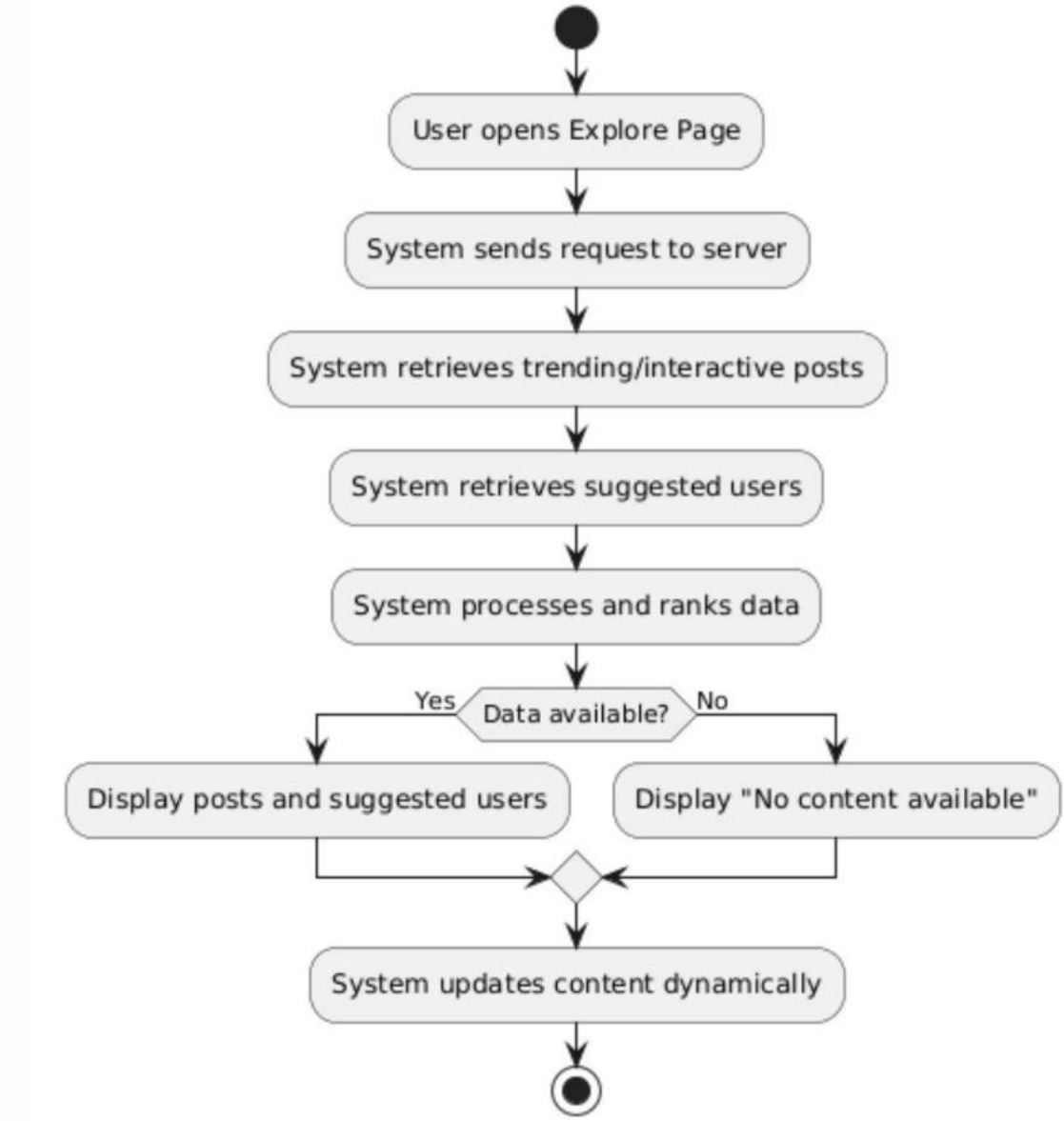

User
Chat / Message Page UI
System / Server
Database
Open chat or Message Page
Request users status
Retrieve users status
Return status data
Send Online / Offline status
alt
[User is Online]
Display "Online"
[User is Offline]
Display "Offline"
Update status dynamically in real time
User
Chat / Message Page UI
System / Server
Database

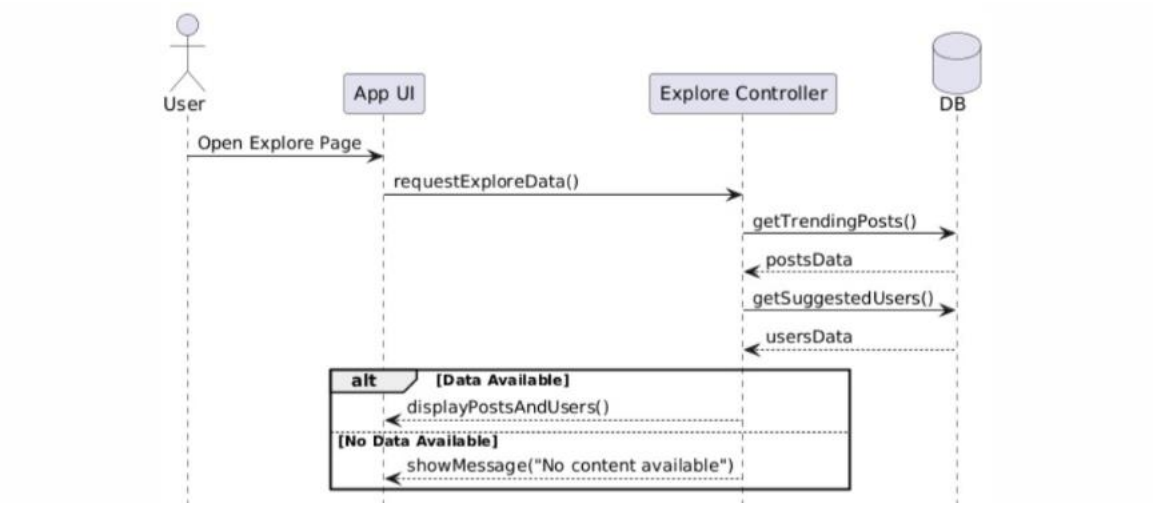
Use Case Name/ID	Search |For Posts Using Hashtags UC 26
Actors	User
Description	This use case allows the user to search for posts using a specific hashtag and view matching results.
Precondition	1. The user must be logged into the system. 2. Posts with Hashtags must already exists in system.
Main flow	1. The user opens the Search Page enters a hashtags and click Search. 2. The system sends the request to the server , retrieves matching posts from the database and displays the results.
Alternative flow	A2: Posts Found : the system displays a list of posts. A2: Posts Found: the system displays “No result found”.
Postconditions	The system displays posts matching the hashtags if no posts are found an appropriate message is shown.


User opens Search Page
Enter hashtag
Click Search
System sends request to server
System retrieves matching posts from database
Yes
Posts found?
No
Display list of posts
Display "No result found"

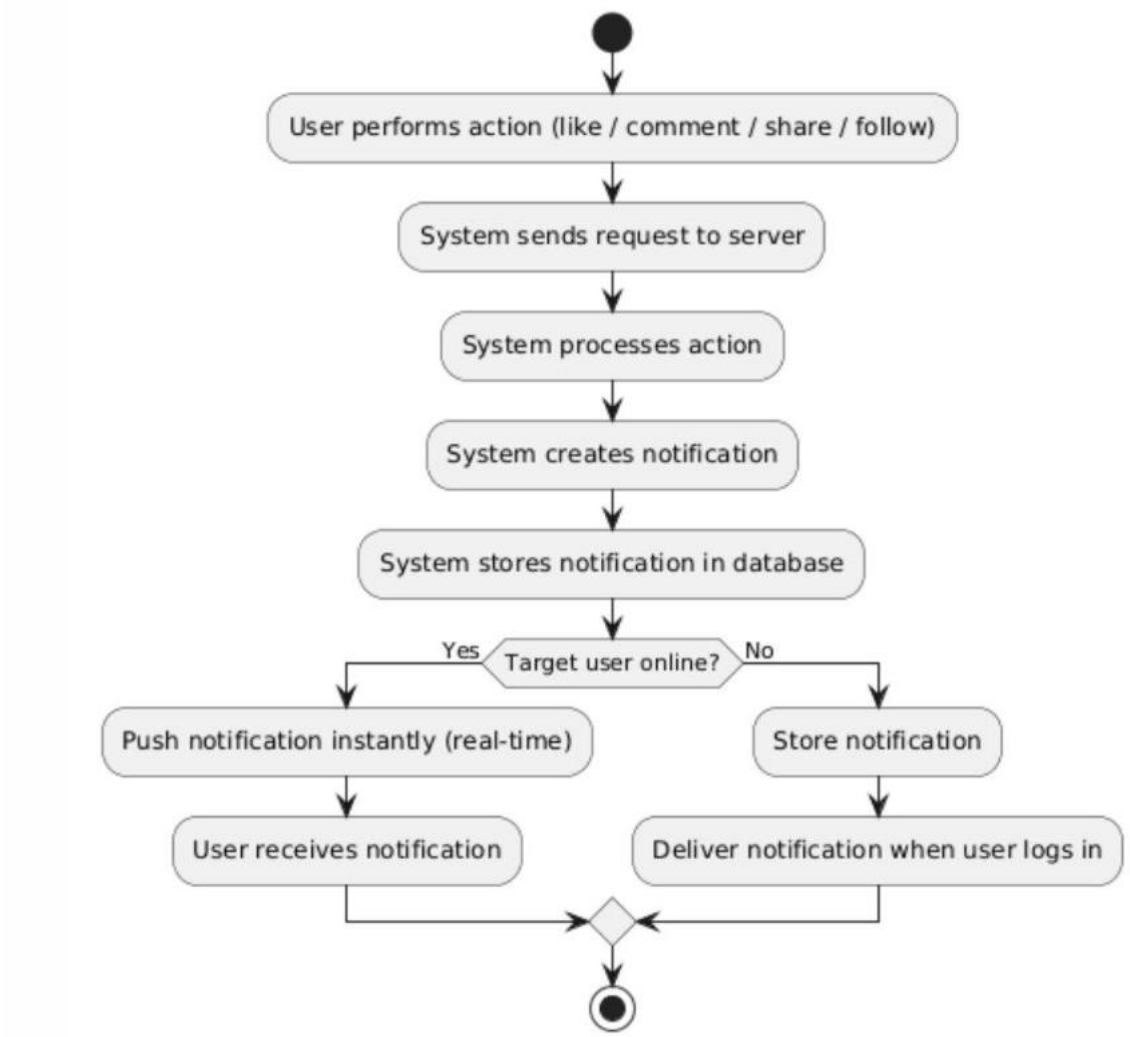

User
App UI
Search Controller
DB
Open Search Page
Enter Hashtag
Click Search
searchByHashtag(hashtag)
getPostsByHashtag(hashtag)
postsData
alt
[Posts Found]
displayPosts()
[No Posts Found]
showMessage("No results found")

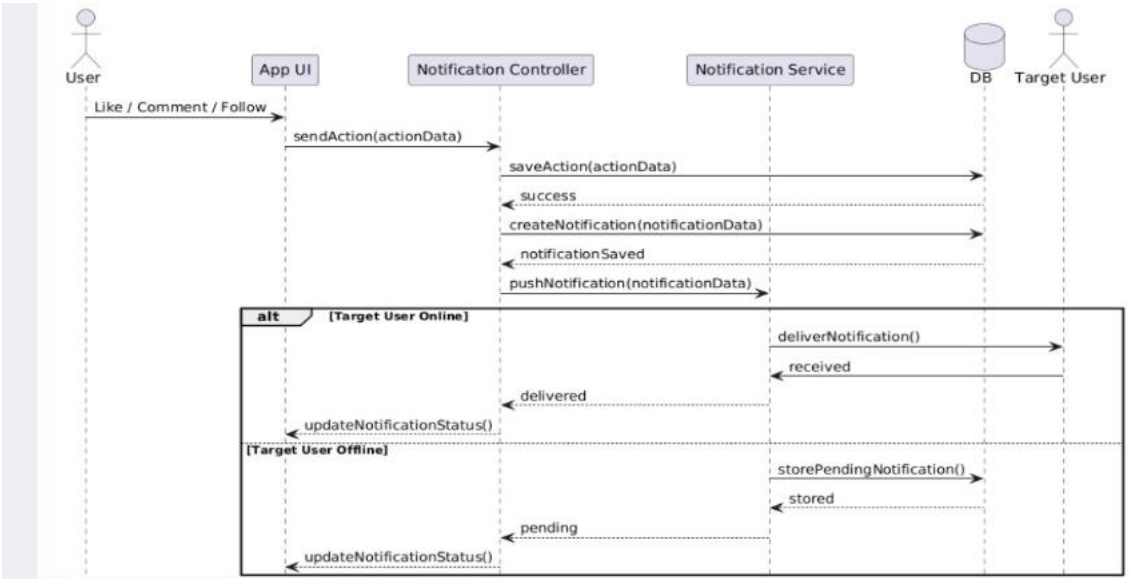
Use Case Name/ID	View Explore Page UC 27
Actors	User
Description	This use case allows the user to view the Explore page, which contains highly interactive posts and a list of suggested users to follow.
Precondition	1. The user must be logged into the system. 2. The system must have posts and user data available.
Main flow	1. The user opens the Explore page. 2. The system sends a request to the server, retrieves trending or interactive posts, retrieves suggested users, processes and ranks the data and displays posts and suggestions.
Alternative flow	A2: Data Available : the system displays posts and suggested users. A2: Data Not Available: the system displays “No content available”.
Postconditions	Interactive posts are displayed , suggested users are shown and the content is updated dynamically.


User opens Explore Page
System sends request to server
System retrieves trending/interactive posts
System retrieves suggested users
System processes and ranks data
Data available?
Yes
No
Display posts and suggested users
Display "No content available"
System updates content dynamically

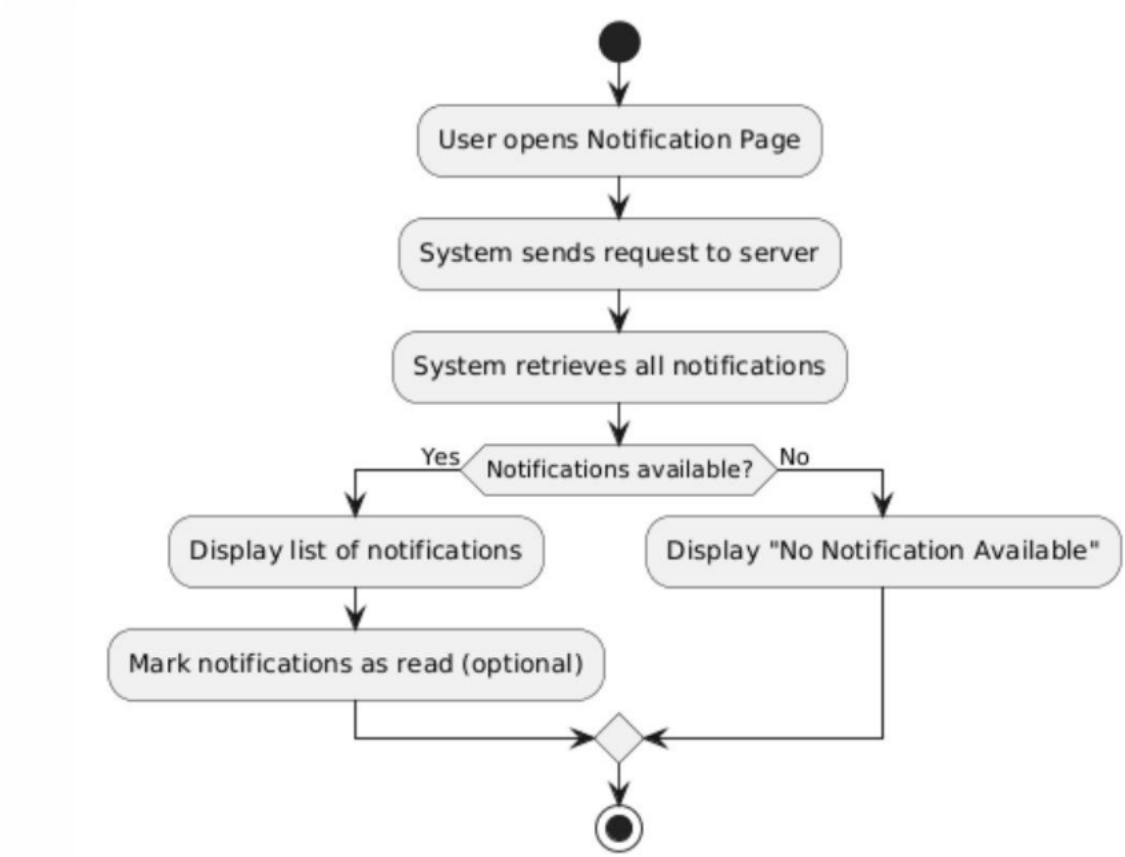


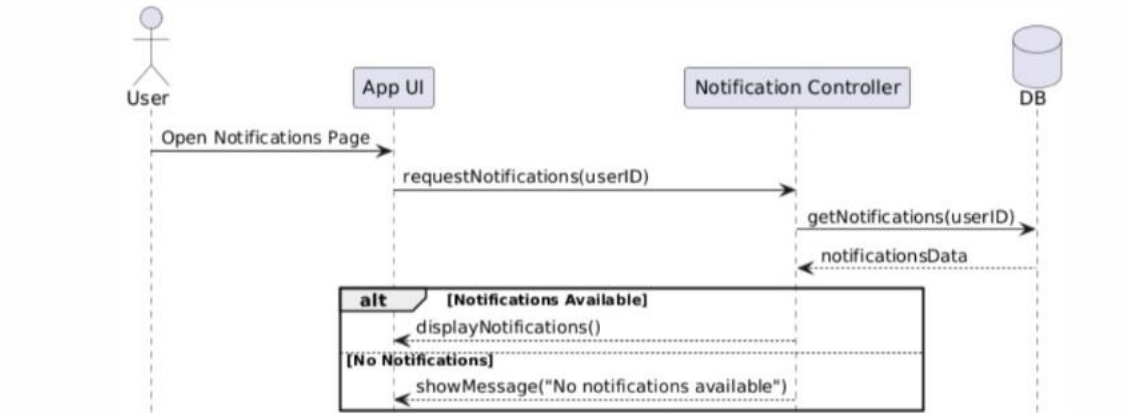
Use Case Name/ID	Send Real-Time Notification UC_28
Actors	User
Description	This use case allows the system to send real-time notifications to user when someone interacts with their posts or new user follows them .
Precondition	1. The user must be logged into the system.
Main flow	1. The user performs an action (like , comment ,share ,follow). 2. The system sends the request to the server , process the action , creates a notification , stores the notification in the database and pushes the notification to the real-time. 3. The target user receives the notification instantly.
Alternative flow	A3: User Online : The system pushes the notification instantly. A3: User Offline : The system stores the notification it receives to the user when they login.
Postconditions	The notification is delivered instantly to the target user and the user sees the notification in real-time .


User performs action (like / comment / share / follow)
System sends request to server
System processes action
System creates notification
System stores notification in database
Target user online?
Yes
No
Push notification instantly (real-time)
Store notification
User receives notification
Deliver notification when user logs in

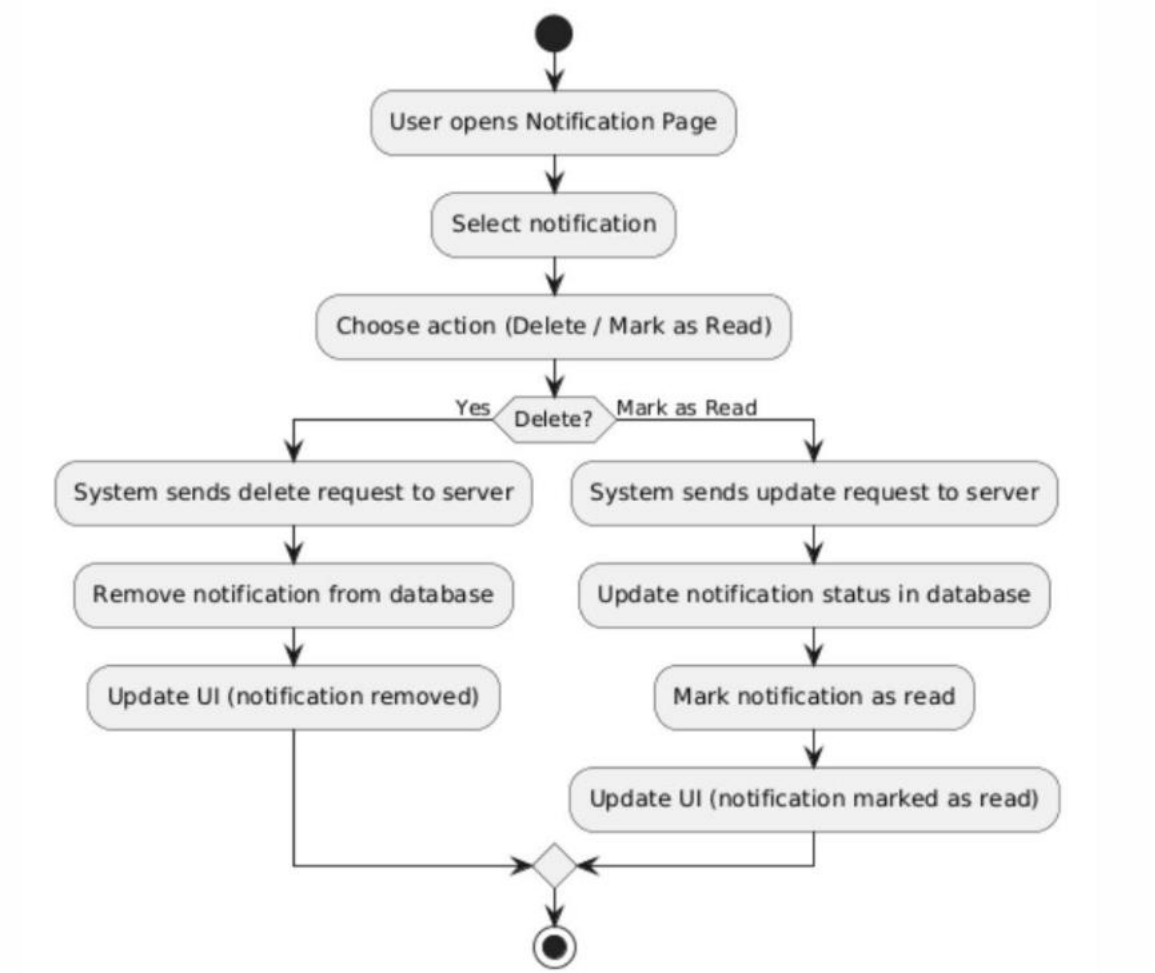

User
App UI
Notification Controller
Notification Service
DB
Target User
Like / Comment / Follow
sendAction(actionData)
saveAction(actionData)
success
createNotification(notificationData)
notificationSaved
pushNotification(notificationData)
alt
[Target User Online]
deliverNotification()
received
delivered
updateNotificationStatus()
[Target User Offline]
storePendingNotification()
stored
pending
updateNotificationStatus()

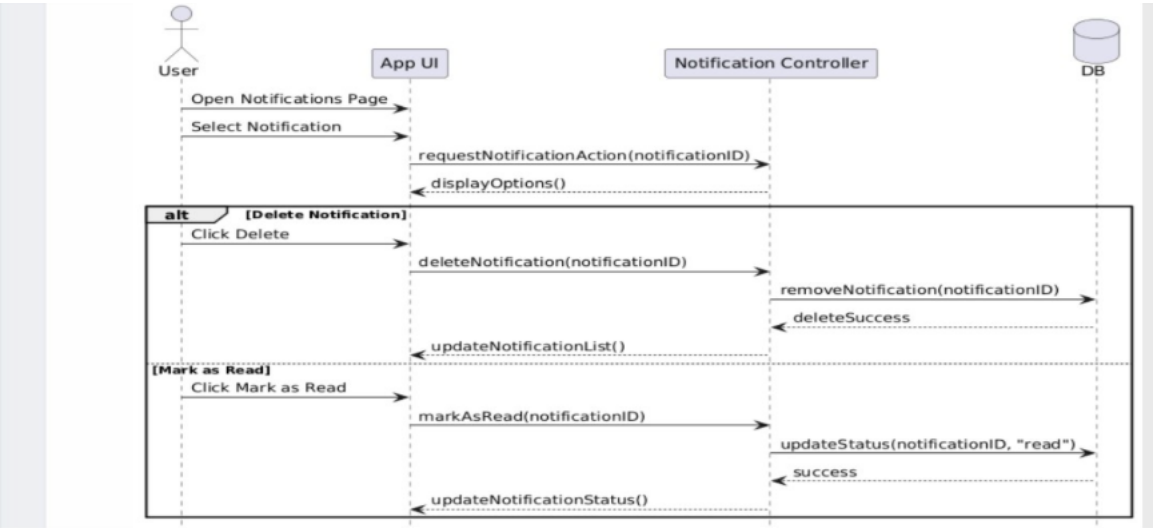
Use Case Name/ID	View All Notification UC 29
Actors	User
Description	This use case allows the user to view all Notification on a dedicated Notification page , including interactions and system alerts .
Precondition	1. The user must be logged into the system. 2. Notification must exists in the system.
Main flow	1. The user opens the Notification page. 2. The system sends the request to the server , retrieves all notifications , displays the notification lists.
Alternative flow	A2: Notification Available : The system displays the list of notifications. A2: Notification Not Available : The system displays “No Notification Available”.
Postconditions	Notification are displayed to the user and Notification may be marked as read .


User opens Notification Page
System sends request to server
System retrieves all notifications
Yes
Notifications available?
No
Display list of notifications
Display "No Notification Available"
Mark notifications as read (optional)

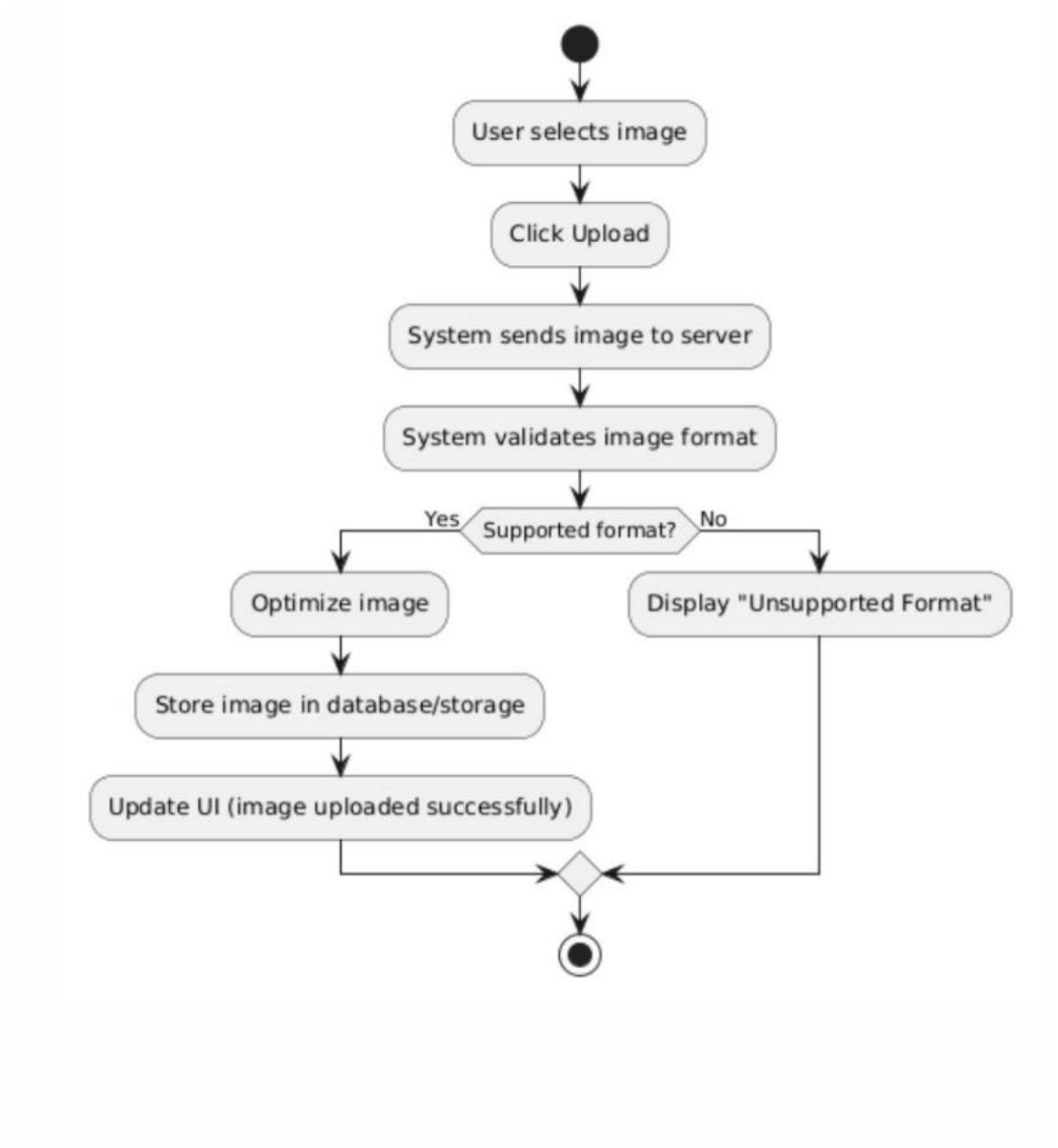

User
App UI
Notification Controller
DB
Open Notifications Page
requestNotifications(userID)
getNotifications(userID)
notificationsData
alt
[Notifications Available]
displayNotifications()
[No Notifications]
showMessage("No notifications available")

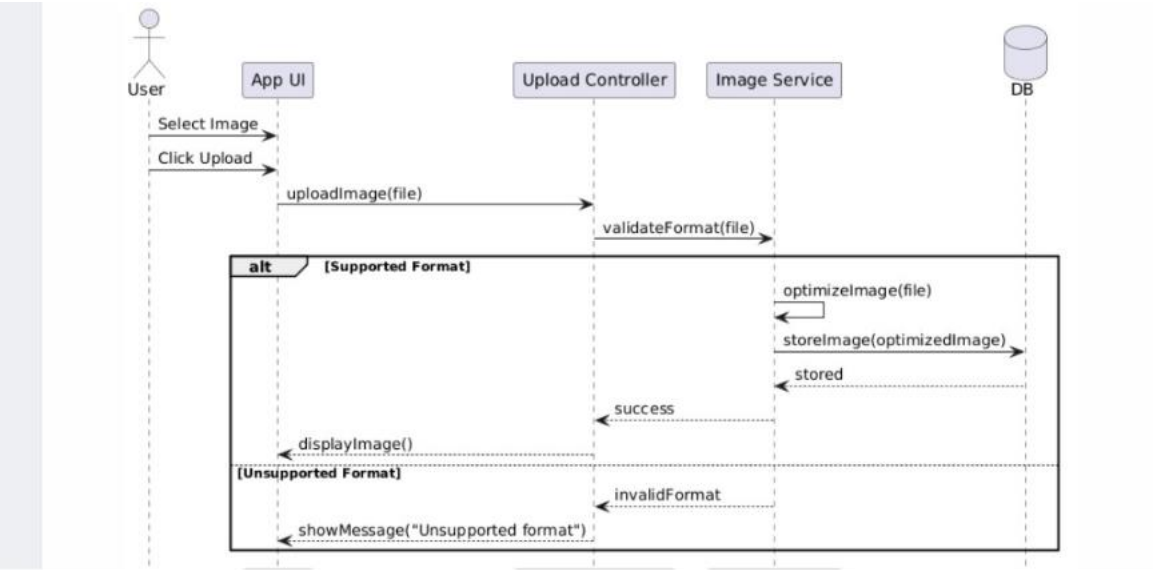
Use Case Name/ID	Delete Notification or Mark as Read UC_30
Actors	User
Description	This use case allows the user to either delete notification or mar k them as read from the notification page.
Precondition	1. The user must be logged into the system. 2. Notification must exists in the system.
Main flow	1. The user opens the Notification page and selects the notification than can choose an action (Delete or Mark as Read). 2. The system sends the request to the server , updates the database and UI.
Alternative flow	A1 : Delete Notification : if the user select delete the system removes the notification from the database and the notification disappears from the list. A2 : if the user select Mark as Read the system updates the notification status and the notification is marked as read
Postconditions	Notification are either deleted or updated as read , UI reflects the updated notification state.


User opens Notification Page
Select notification
Choose action (Delete / Mark as Read)
Yes
Delete?
Mark as Read
System sends delete request to server
Remove notification from database
Update UI (notification removed)
System sends update request to server
Update notification status in database
Mark notification as read
Update UI (notification marked as read)


User
App UI
Notification Controller
DB
Open Notifications Page
Select Notification
requestNotificationAction(notificationID)
displayOptions()
alt
[Delete Notification]
Click Delete
deleteNotification(notificationID)
removeNotification(notificationID)
deleteSuccess
updateNotificationList()
[Mark as Read]
Click Mark as Read
markAsRead(notificationID)
updateStatus(notificationID, "read")
success
updateNotificationStatus()

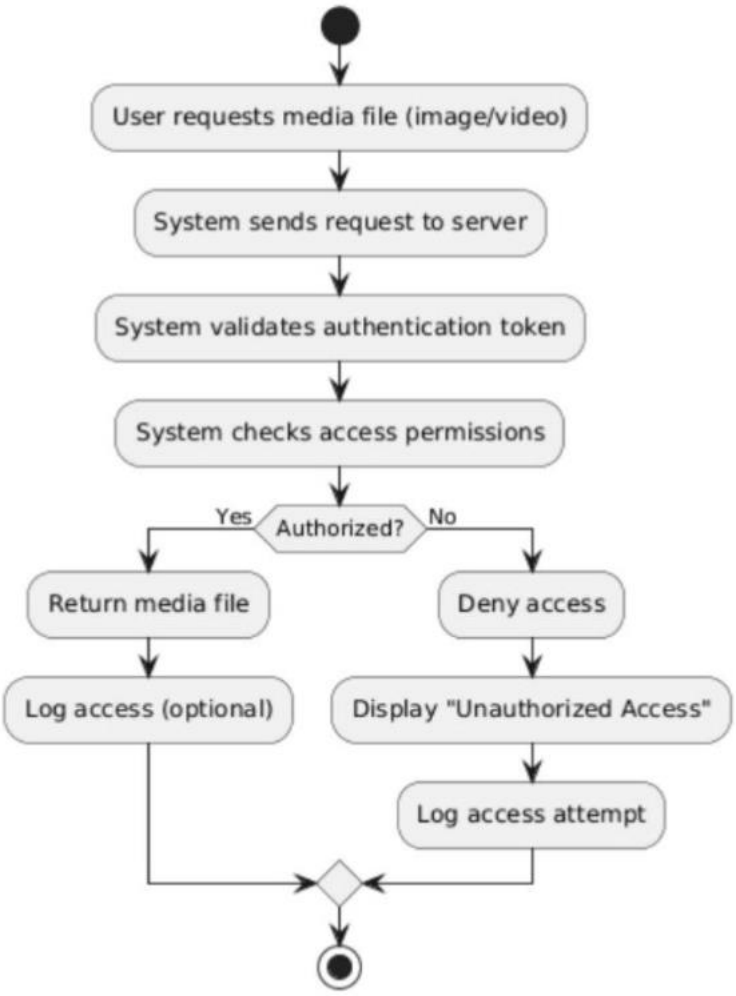
Use Case Name/ID	Upload and Optimize |Images UC_31
Actors	User
Description	This use case allows the user to upload the images , and the system automatically optimizes them and support multiple format.
Precondition	1. The user must be logged into the system. 2. The image file must be selected .
Main flow	1. The user selects an image , clicks Upload. 2. The system sends the image to the server , validates the format , optimizes and store the image . 3. The system updates the UI.
Alternative flow	A2 |: Supported Format : The system process and optimize the images. A2 : Unsupported Image : The system displays “Unsupported Format”.
Postconditions	The image is uploaded successfully , stored and optimized for display.

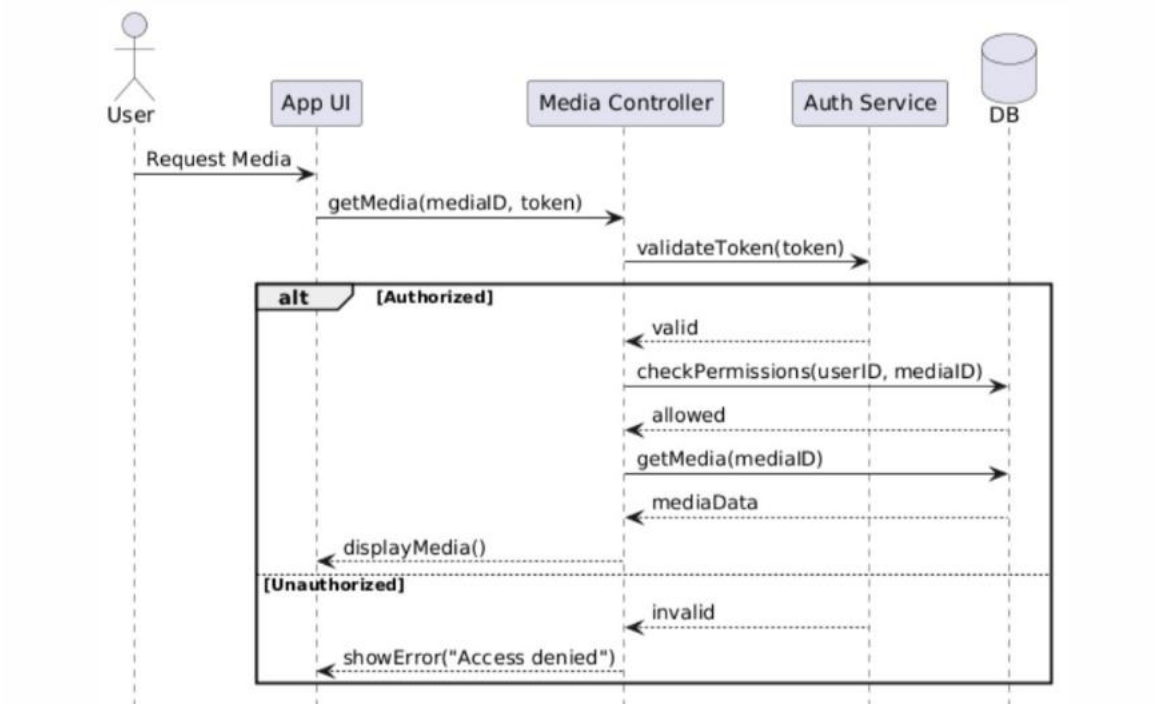

User selects image
Click Upload
System sends image to server
System validates image format
Supported format?
Yes
No
Optimize image
Display "Unsupported Format"
Store image in database/storage
Update UI (image uploaded successfully)



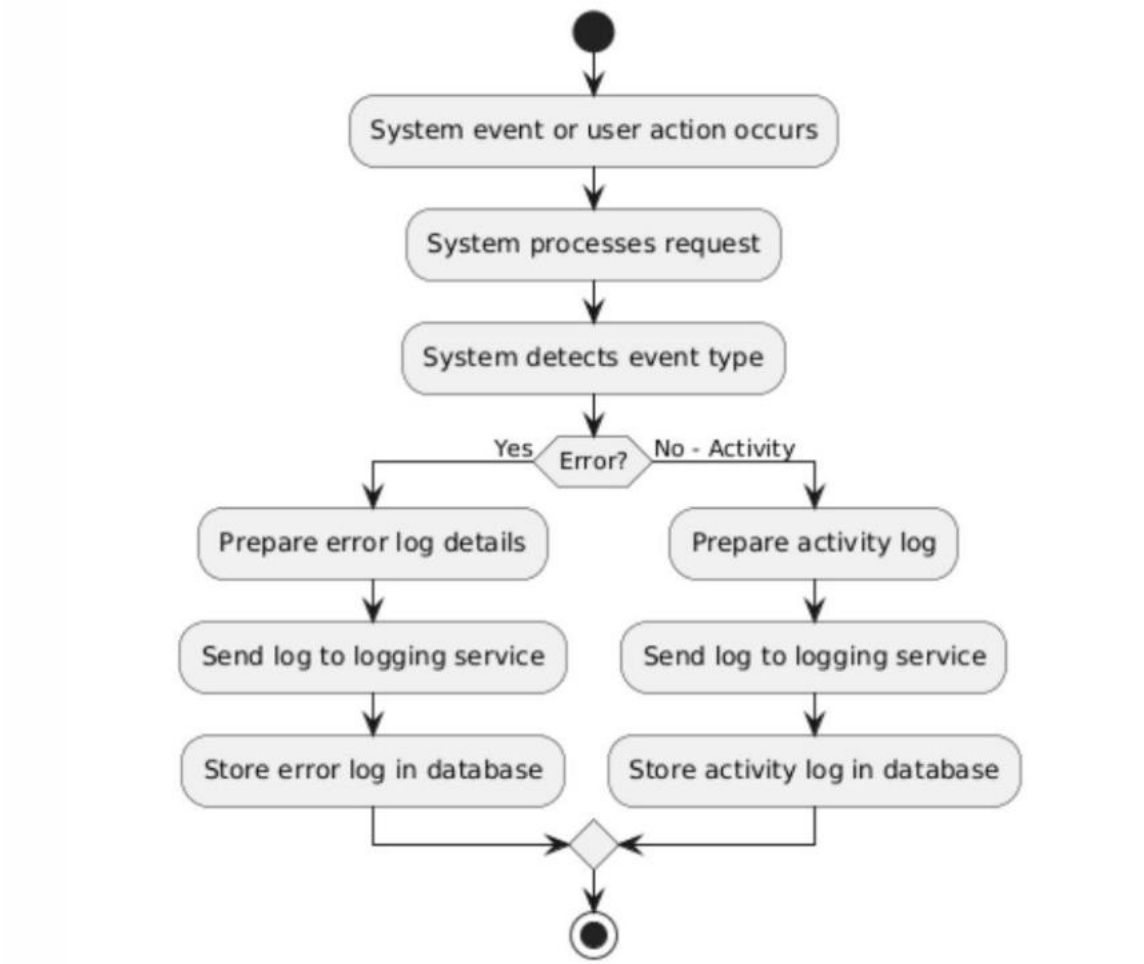
Use Case Name/ID	Protect Media Links From Unauthorized Access UC_32

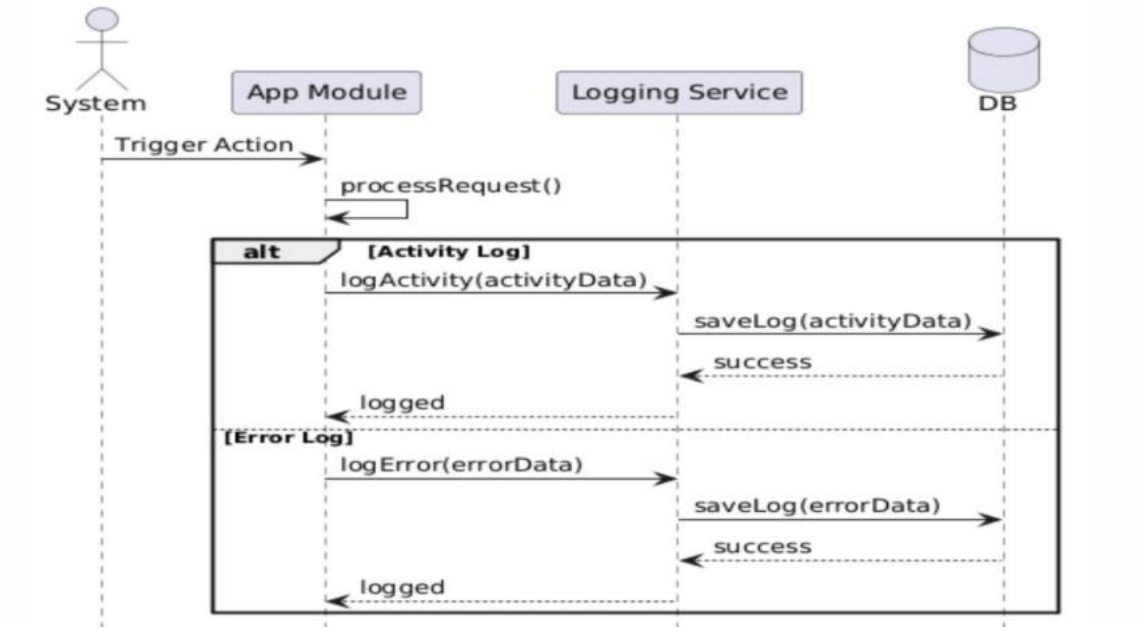
Actors	User
Description	This use case ensures that media files (images,videos) are accessible only to authorized users by validating access permissions before allowing access.
Precondition	1. The user must be logged into the system. 2. The image file must be selected . 3. The user must have permission to access the media.
Main flow	1. The user requests a media file (image/video). 2. The system sends the request to the serve ,validates the user's authentication token, checks access permissions and allows access to the media.
Alternative flow	A2: Authorized Access :The system returns the media file. A2: Unauthorized Access :The system denies access and shows an error message.
Postconditions	Authorized users can access media , Unauthorized users are denied access. Access attempts may be logged for security.


User requests media file (image/video)
System sends request to server
System validates authentication token
System checks access permissions
Authorized?
Yes
No
Return media file
Deny access
Log access (optional)
Display "Unauthorized Access"
Log access attempt

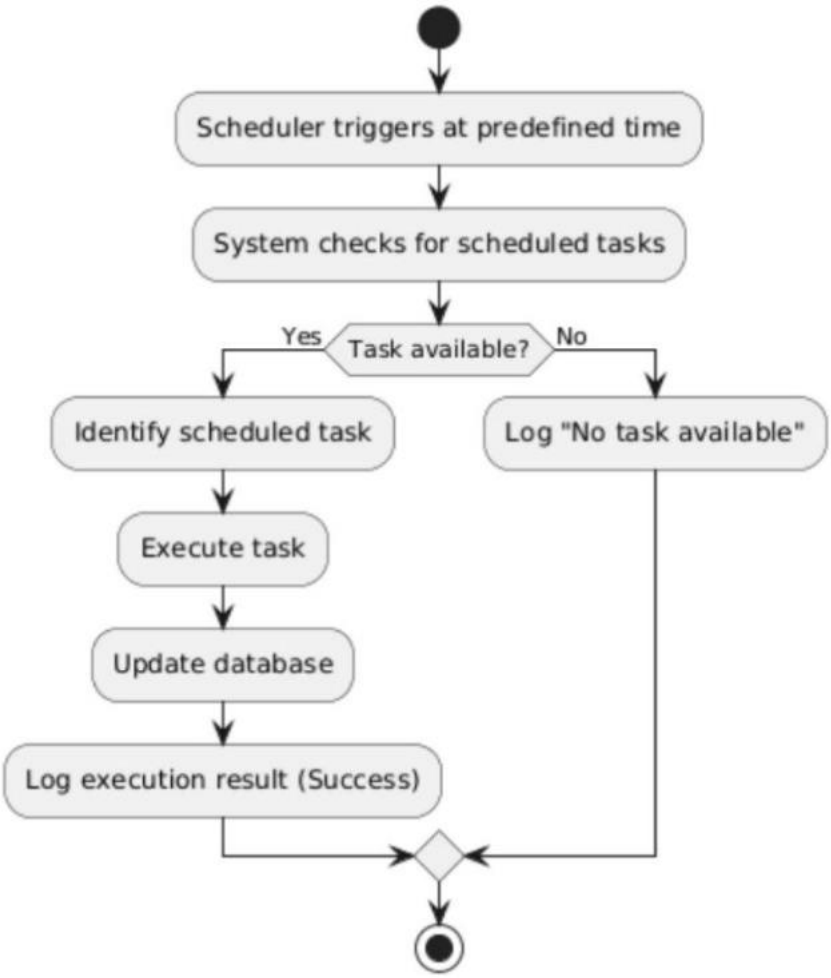


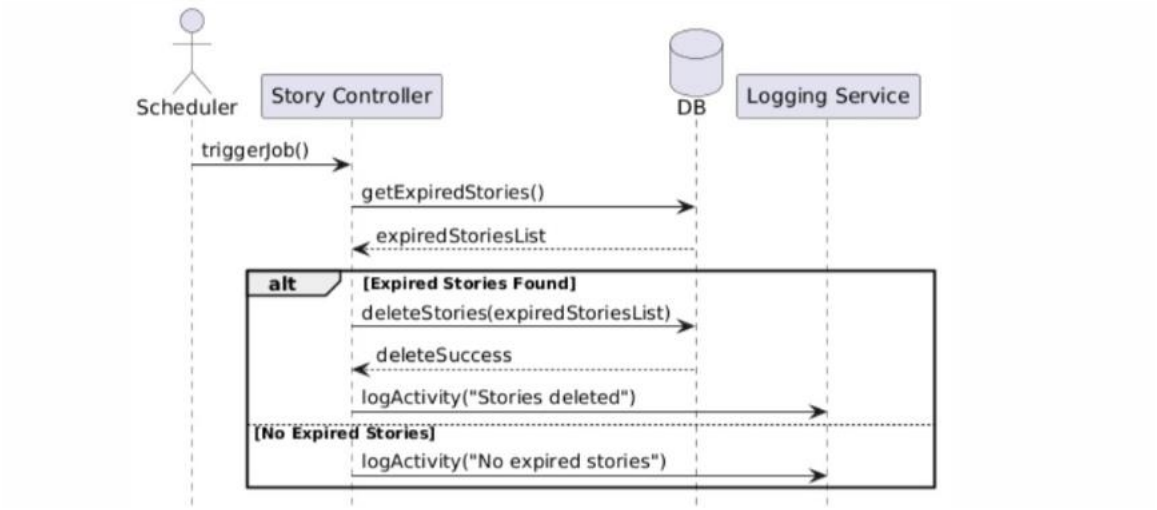
Use Case Name/ID	Log Errors and System Activities UC_33
Actors	User
Description	This use case allows the system to log errors and system activities for monitoring, debugging, and auditing purposes.
Precondition	1. The system is running. 2. Logging service is available.
Main flow	1. A system event or user action occurs. 2. The system processes the request , detects an event (error or activity) , sends log data to the logging service. 3. The logging service stores the log in the database.
Alternative flow	A3 |: Activity Log : The system records normal system activity. A3 : Error Log : The system records error details.
Postconditions	Errors and activities are recorded in the system logs and Logs can be reviewed for debugging or auditing.


System event or user action occurs
System processes request
System detects event type
Error?
Yes
No - Activity
Prepare error log details
Send log to logging service
Store error log in database
Prepare activity log
Send log to logging service
Store activity log in database


System
App Module
Logging Service
DB
Trigger Action
processRequest()
alt
[Activity Log]
logActivity(activityData)
saveLog(activityData)
success
logged
[Error Log]
logError(errorData)
saveLog(errorData)
success
logged

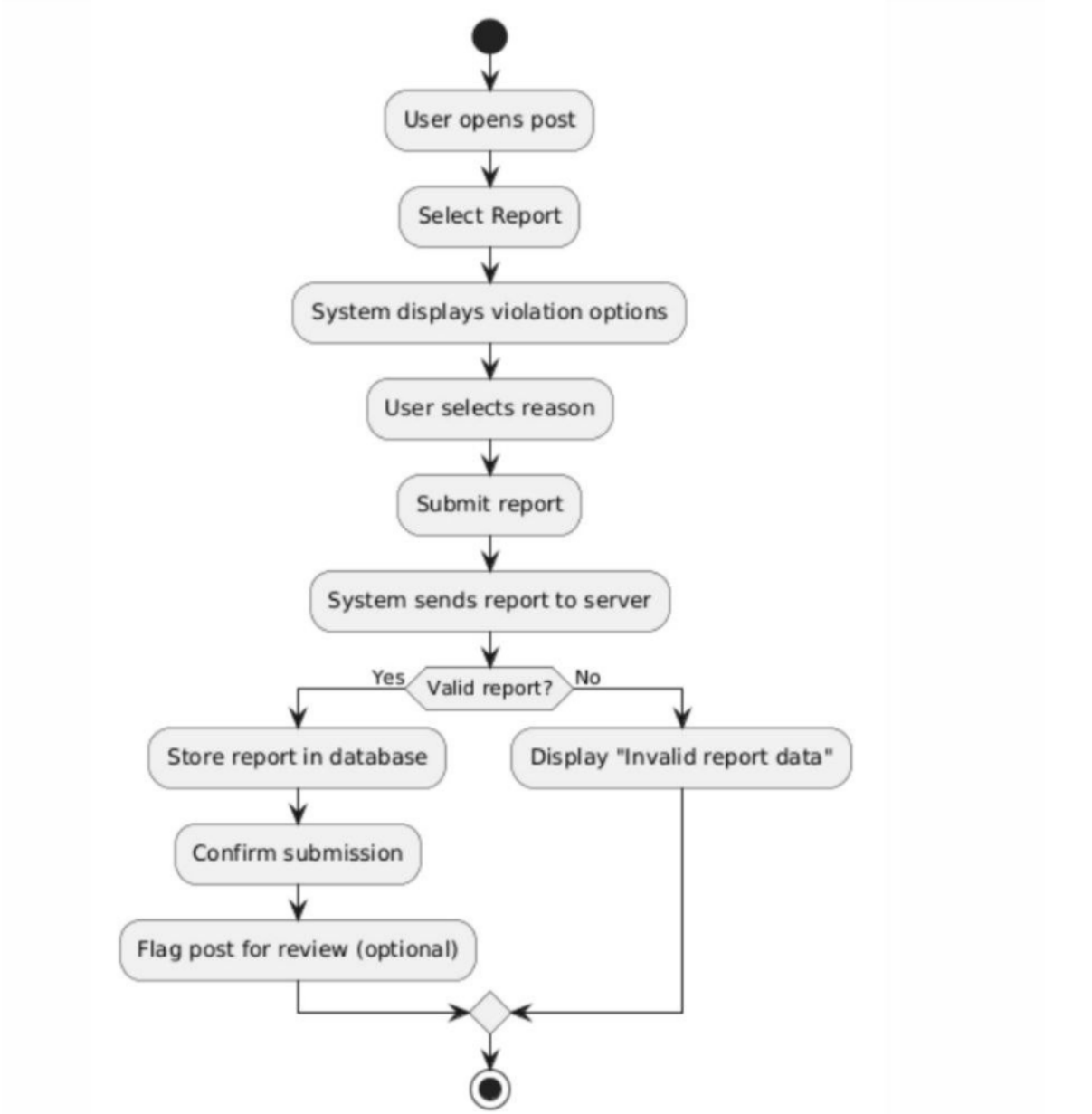
Use Case Name/ID	Execute Scheduled Tasks UC 34
Actors	User
Description	This use case allows the system to automatically execute scheduled tasks at predefined times (e.g., deleting expired data, sending reminders, updating status.
Precondition	1. The system is running. 2. The scheduler service is active. 3. Scheduled tasks are predefined.
Main flow	1. The scheduler triggers at a predefined time. 2. The system identifies the scheduled task, executes the task , updates the database and logs the execution result.
Alternative flow	A2 : Task Executed Successfully : The system completes the task and logs success. A2: No Task to Execute : The system does nothing and logs "No task available".
Postconditions	The scheduled task is executed successfully. The system data is updated accordingly and logs the execution result.

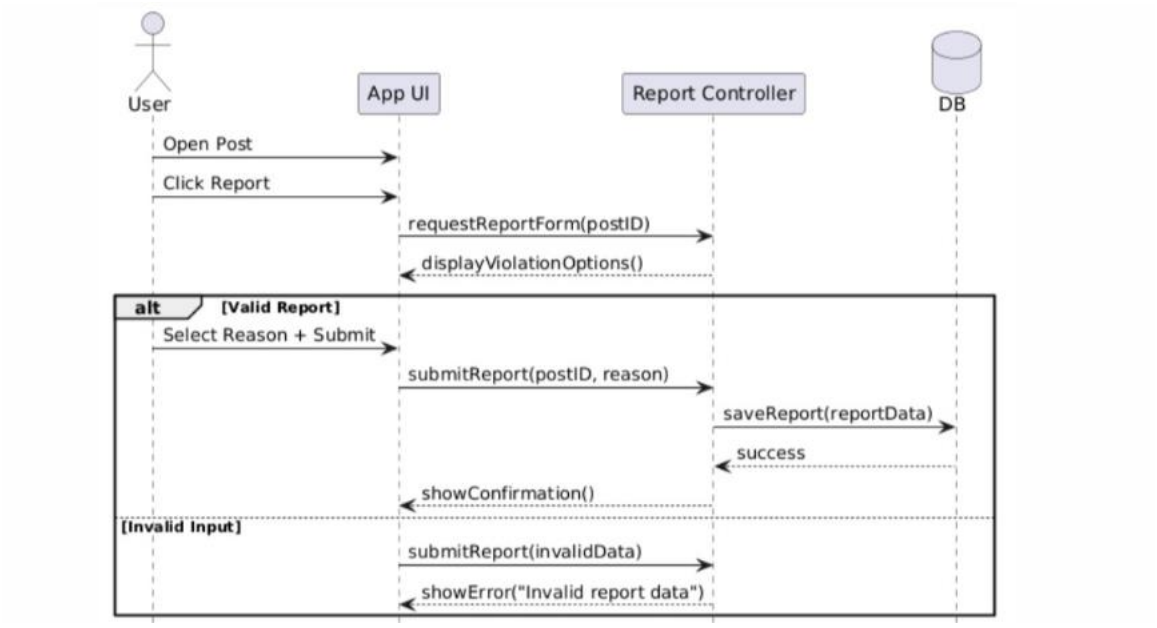

Scheduler triggers at predefined time
System checks for scheduled tasks
Task available?
Yes
No
Identify scheduled task
Log "No task available"
Execute task
Update database
Log execution result (Success)



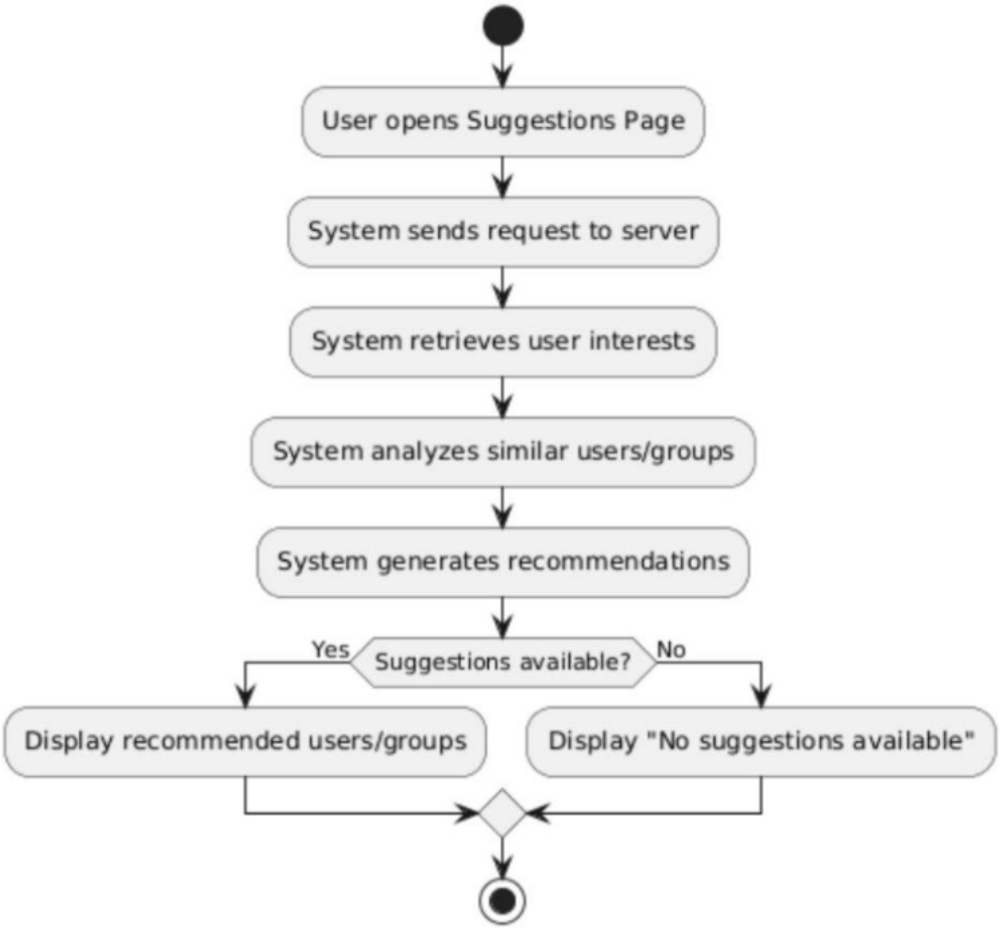
Use Case Name/ID	Report Inappropriate or Violating Posts UC_35

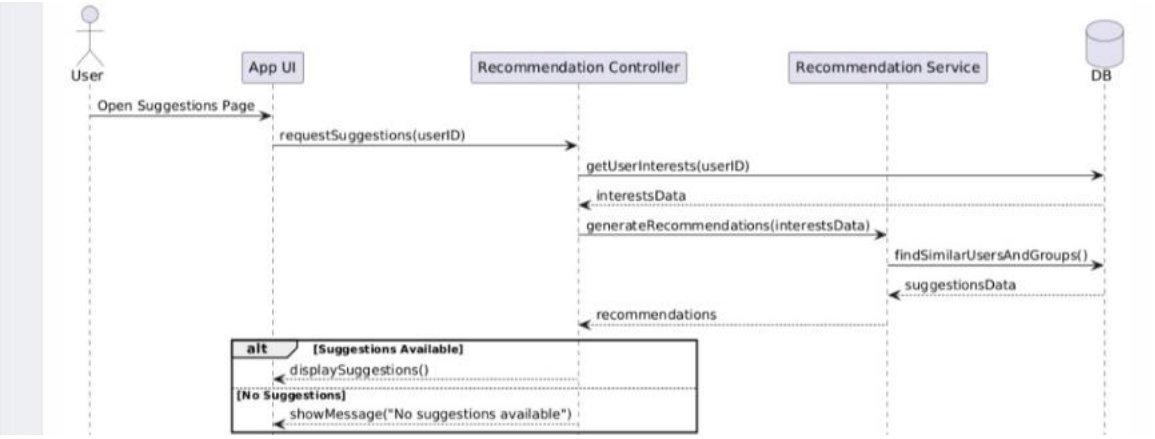
Actors	User
Description	This use case allows the user to report posts that violate platform rules (e.g., spam, abuse, inappropriate content).
Precondition	1. The user must be logged into the system. 2. The post must exist.
Main flow	1. The user opens a post , selects Report. 2. The system displays violation options. 3. The user selects a reason and submits. 4. The system sends the report to the server , stores the report in the database and confirms submission.
Alternative flow	A4 : A1: Valid Report : The system saves the report successfully. A4: Invalid Input :The system shows "Invalid report data
Postconditions	The report is saved in the system and the system may flag the post for review.


User opens post
Select Report
System displays violation options
User selects reason
Submit report
System sends report to server
Valid report?
Yes
No
Store report in database
Display "Invalid report data"
Confirm submission
Flag post for review (optional)

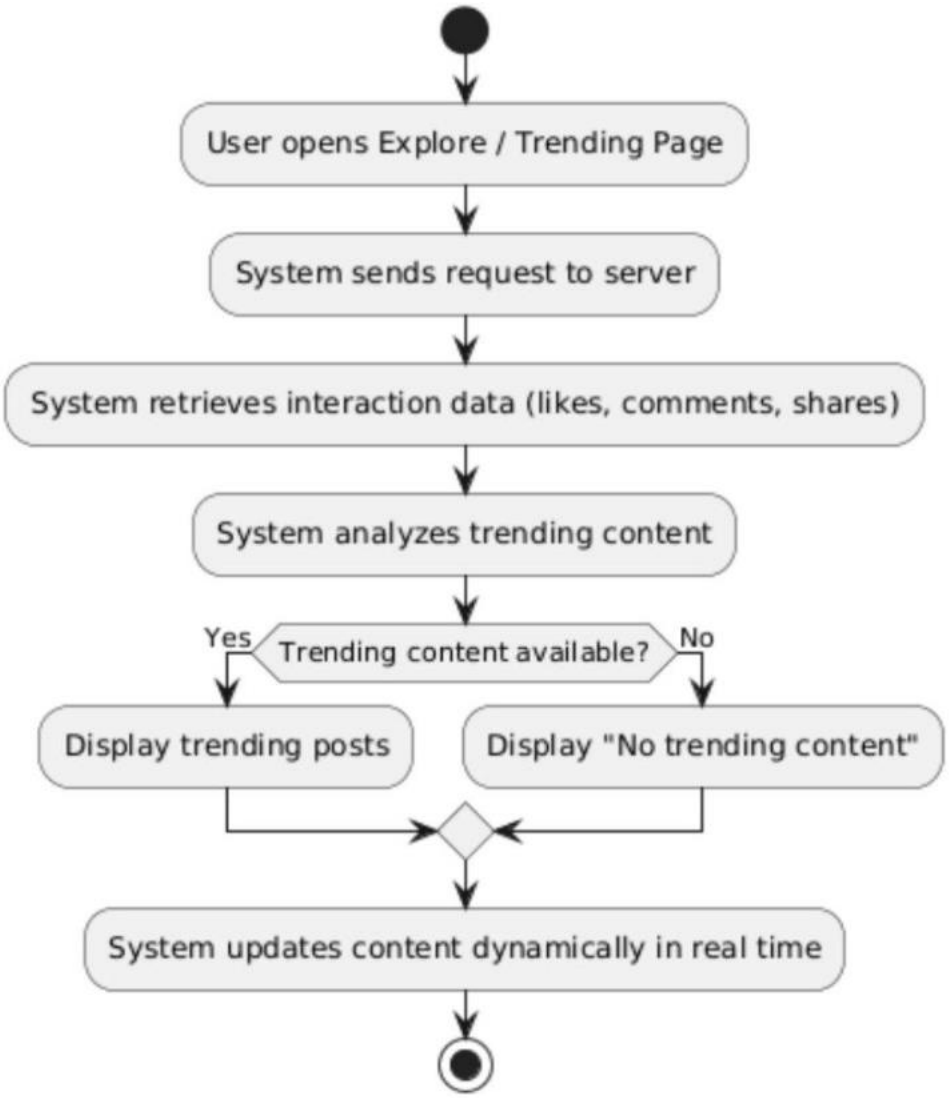


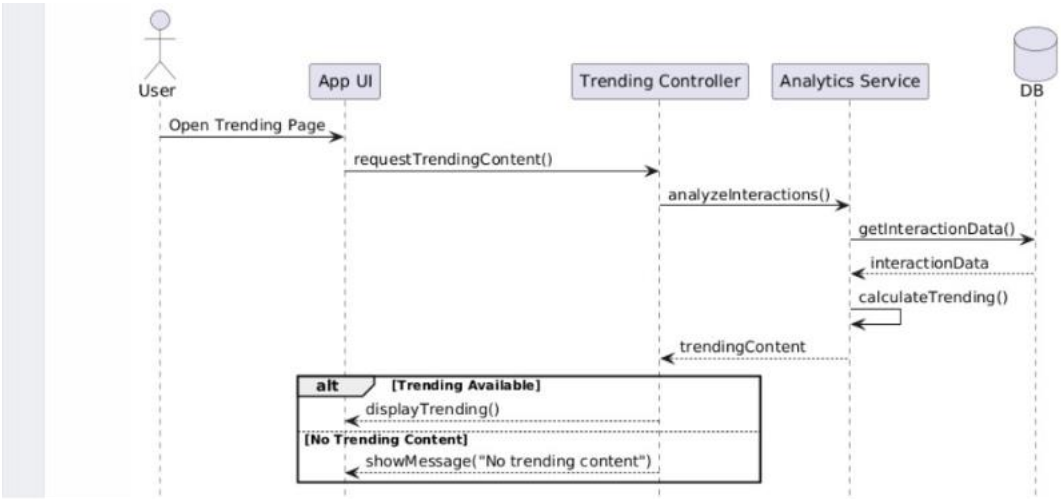
Use Case Name/ID	Suggest Friends or Groups Based on Shared Interests UC_36
Actors	User
Description	This use case allows the system to suggest friends or groups to the user based on shared interests, activities, or interactions.
Precondition	1. The user must be logged into the system. 2. User profile and interests must exist.
Main flow	1. The user opens the suggestions page. 2. The system sends a request to the server , retrieves user interests , analyzes similar users/groups , generates recommendations and displays suggestions.
Alternative flow	A2: Suggestions Available The system displays recommended users/groups. A2: No Suggestions The system displays "No suggestions available".
Postconditions	Relevant suggestions are displayed. Suggestions are based on similarity or system recommendations.


User opens Suggestions Page
System sends request to server
System retrieves user interests
System analyzes similar users/groups
System generates recommendations
Suggestions available?
Yes
No
Display recommended users/groups
Display "No suggestions available"



Use Case Name/ID	Discover Trending Content in Real Time UC_37
Actors	User
Description	This use case allows the user to discover trending content in real time based on interactions such as likes, comments, and shares..
Precondition	1. The user must be logged into the system. 2. Content and user interactions must exist. 3. Real-time or analytics service must be active.
Main flow	1. The user opens the Explore / Trending page. 2. The system sends a request to the server, retrieves interaction data (likes, comments, shares). 3. The system analyzes trending content , sends results to the UI and updates content dynamically.
Alternative flow	A2: Trending Content Available The system displays trending posts. A2: No Trending Content The system displays "No trending content".
Postconditions	Trending content is displayed. Content updates dynamically in real time.


User opens Explore / Trending Page
System sends request to server
System retrieves interaction data (likes, comments, shares)
System analyzes trending content
Yes
Trending content available?
No
Display trending posts
Display "No trending content"
System updates content dynamically in real time


User
App UI
Trending Controller
Analytics Service
DB
Open Trending Page
requestTrendingContent()
analyzeInteractions()
getInteractionData()
interactionData
calculateTrending()
trendingContent
alt
[Trending Available]
displayTrending()
[No Trending Content]
showMessage("No trending content")

Chapter Three: Design Study

Site Map Diagram

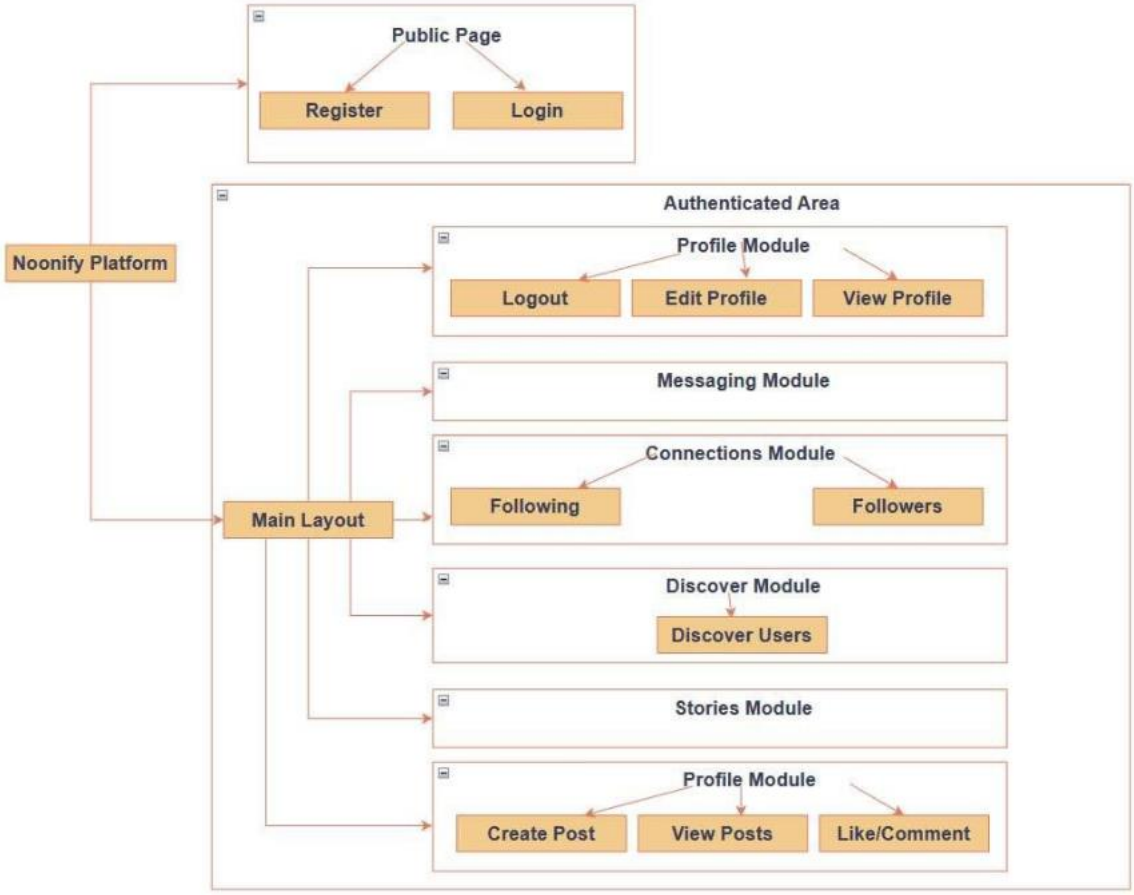


Figure : Sit Map Diagram

Database ERD Diagram (Entity-Relationship Diagram)

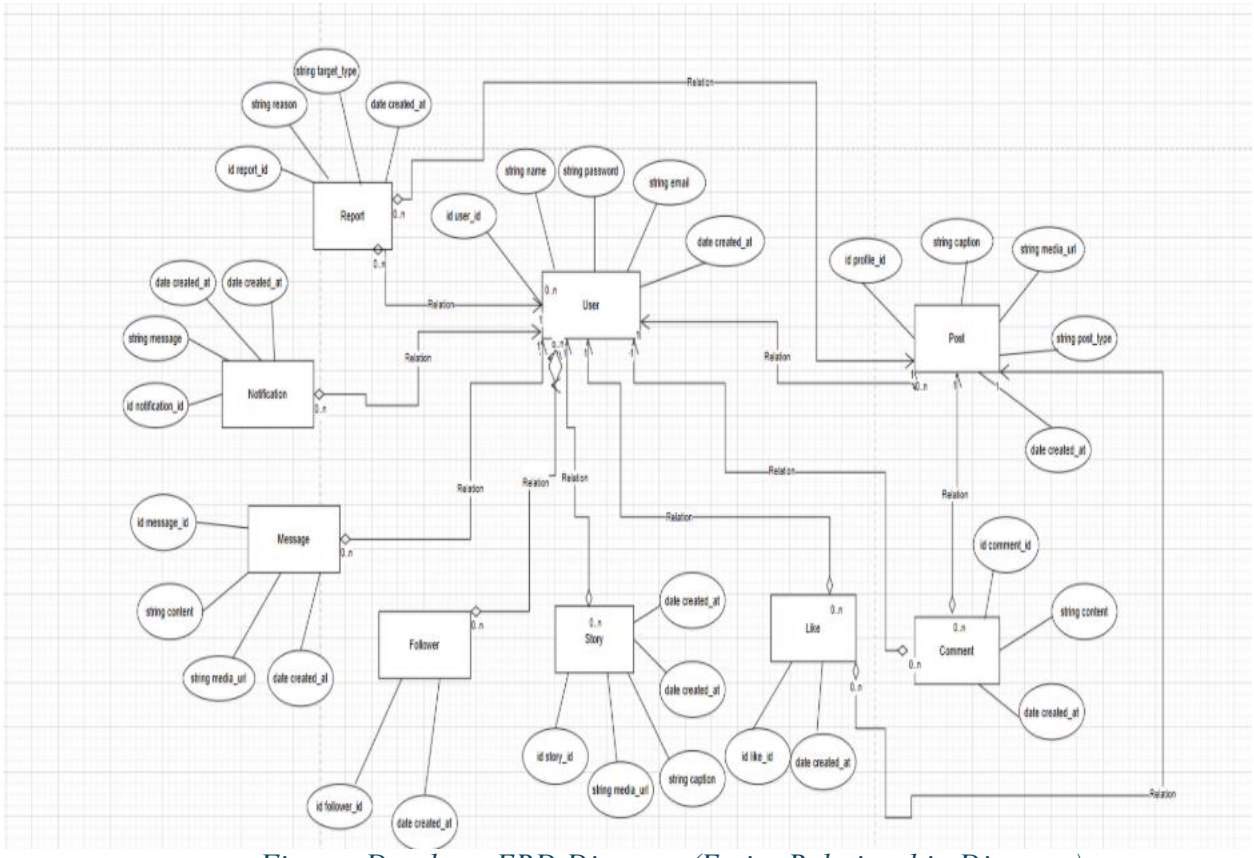


Figure: Database ERD Diagram (Entity-Relationship Diagram)

Class Diagram

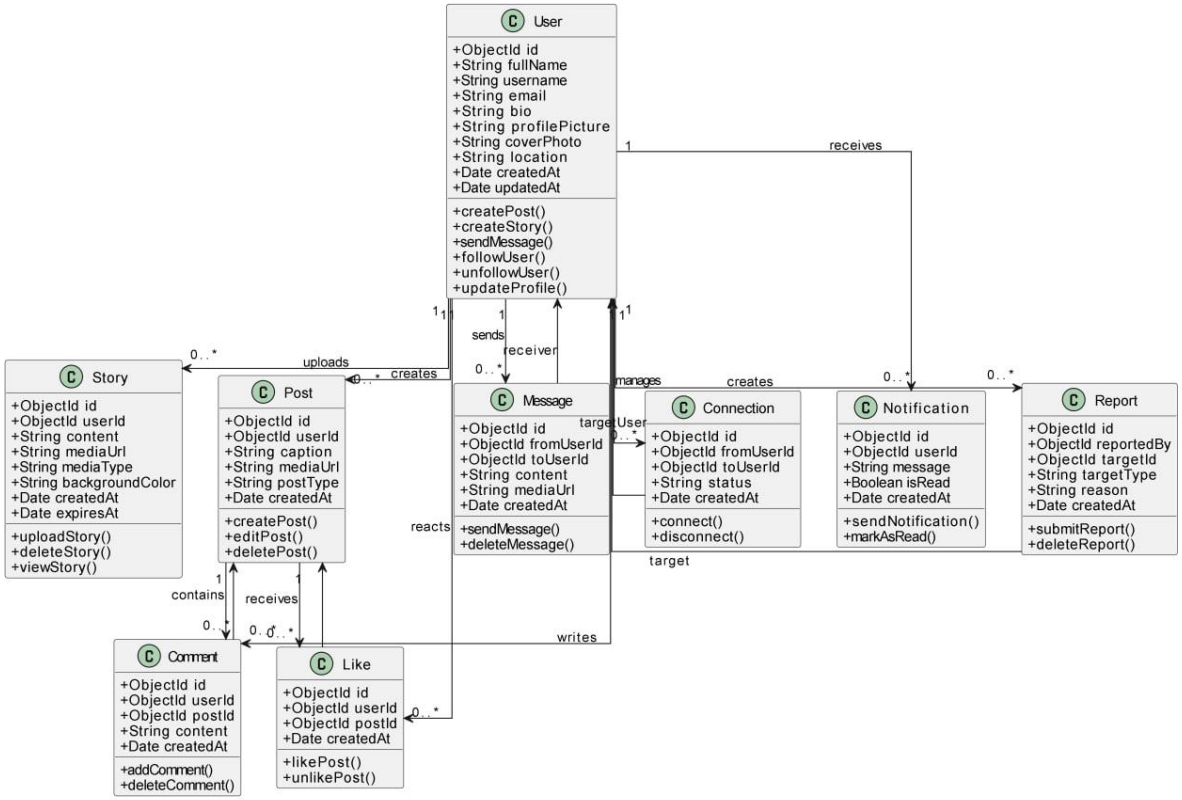


Figure: Class Diagram

Test Cases Table

Test Case ID	Test Case Description	Preconditions	Test Steps	Expected Result

TC-01	Start a new conversation between users	Users are logged in	Open chat → Select user → Start conversation	Conversation is created successfully
TC-02	Send and receive text messages and emojis	Active conversation exists	Type message → Send	Message is delivered and displayed correctly
TC-03	Support real-time messaging	Users online	Send message	Message appears instantly for receiver
TC-04	Display message status (Sent – Delivered – Read)	Message sent	Observe message status	Status updates correctly
TC-05	Delete or mute conversations	Conversation exists	Select chat → Delete/Mute	Chat is deleted or muted successfully
TC-06	Follow or unfollow users	User logged in	Open profile → Click Follow/Unfollow	Follow status updated
TC-07	Block or unblock users	User logged in	Open profile → Block/Unblock	User is blocked/unblocked successfully
TC-08	Display online/offline status	Users exist	View user list	Correct status is displayed
TC-09	Search for posts using hashtags	Posts exist	Enter hashtag → Search	Relevant posts displayed
TC-10	View Explore page	User logged in	Open Explore page	Suggested users and posts displayed
TC-11	Send real-time notifications	Interaction occurs	Like/Follow	Notification sent instantly
TC-12	View all notifications	Notifications exist	Open notifications page	All notifications displayed
TC-13	Delete or mark notifications as read	Notifications exist	Select notification → Delete/Mark	Notification updated correctly
TC-14	Upload and optimize images	User logged in	Upload image	Image uploaded and optimized
TC-15	Protect media links	Media uploaded	Access link without permission	Access denied
TC-16	Log errors and activities	System running	Trigger error	Error is logged

TC-17	Execute scheduled tasks	Stories exist	Wait 24h	Expired stories deleted
TC-18	Report inappropriate posts	Post exists	Click Report	Report submitted
TC-19	Suggest friends/groups	Users exist	Open suggestions	Relevant suggestions displayed
TC-20	Discover trending content	Content exists	Open explore	Trending content displayed

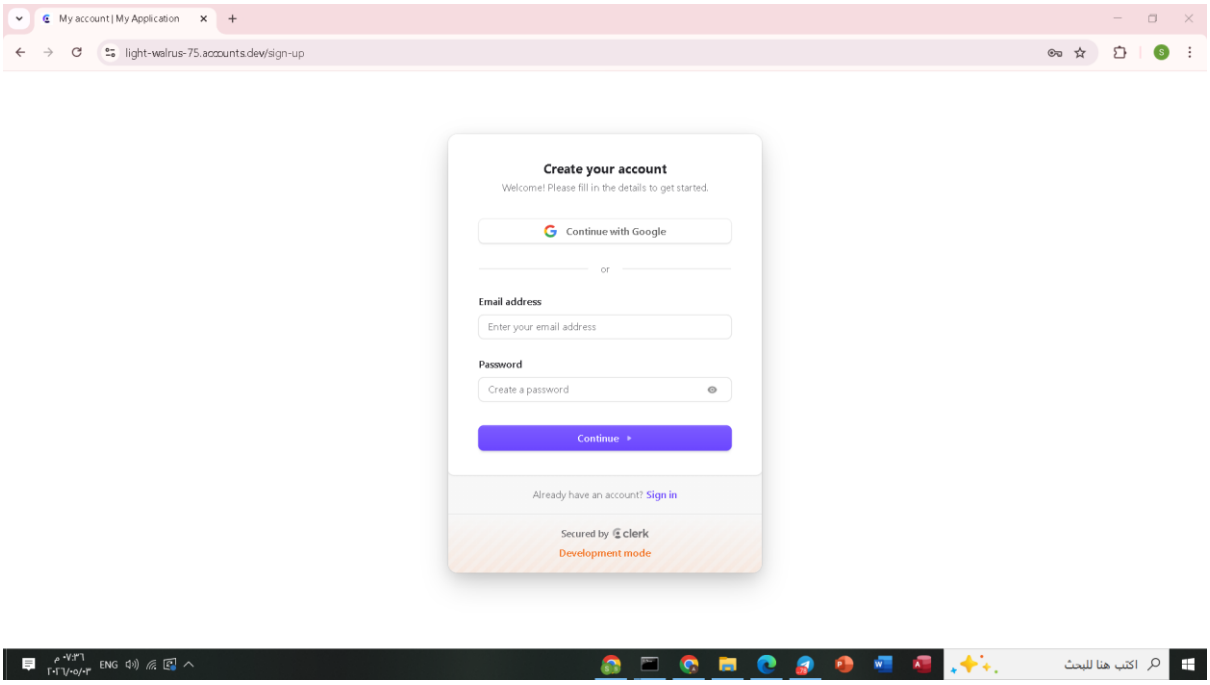
Requirements Traceability Matrix (RTM)

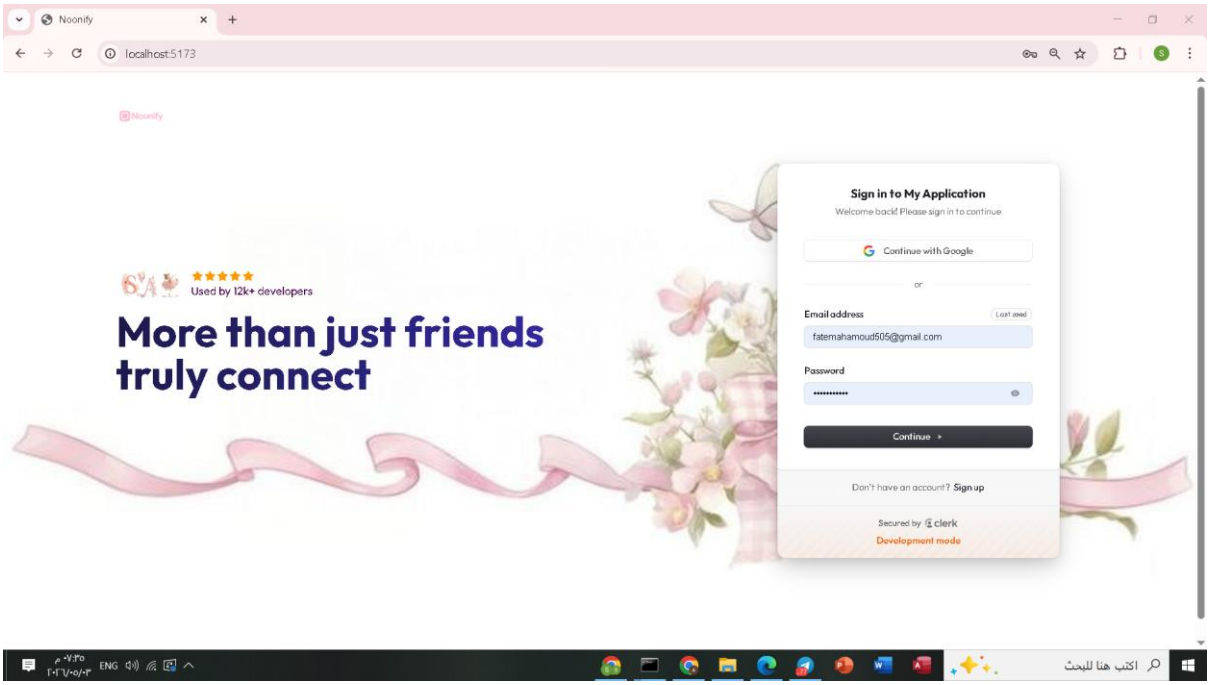
Req ID	Requirement Description	Use Case	Design	Design Description	Implementation	Test Case ID
ID-01	Start a new conversation between users	Start Chat Use Case	Chat UI Design	User selects a user and initiates chat	Message Module Developed	TC-01
ID-02	Send and receive text messages and emojis	Messaging Use Case	Chat Input Design	Message input and send mechanism	Message Service Developed	TC-02
ID-03	Support real-time messaging	Real-time Chat Use Case	Socket Design	Live message transmission using sockets	Socket Service Developed	TC-03
ID-04	Display message status (Sent–Delivered–Read)	Message Status Use Case	Status UI Design	Track and display message states	Message Status Module Developed	TC-04
ID-05	Delete or mute conversations	Conversation Control Use Case	Settings Design	User can mute or delete conversations	Message Control Module Developed	TC-05
ID-06	Follow or unfollow users	Follow System Use Case	User Interaction Design	Follow/unfollow mechanism	Connection Module Developed	TC-06
ID-07	Block or unblock users	Block System Use Case	Privacy Control Design	User blocking functionality	Security Module Developed	TC-07
ID-08	Display online/offline status	Presence Use Case	Status Indicator Design	Show user availability status	Presence Service Developed	TC-08
ID-09	Search posts using hashtags	Search Use Case	Search UI Design	Hashtag-based filtering system	Search Module Developed	TC-09

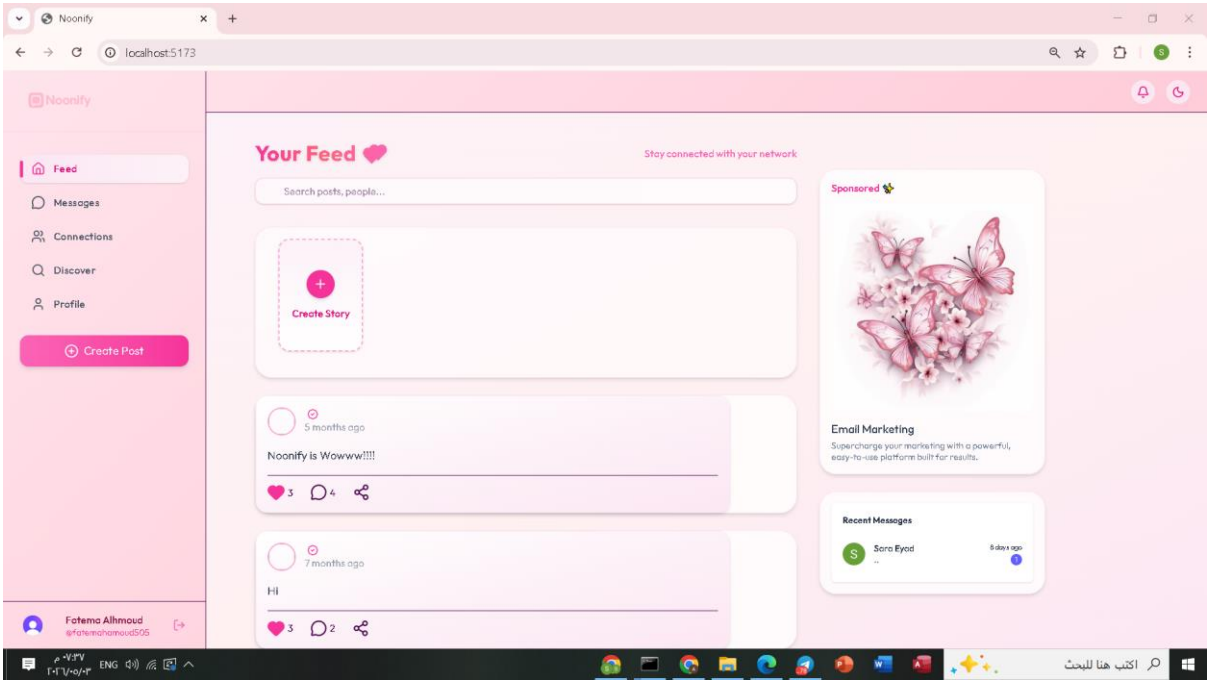
ID-10	View Explore page (trending content)	Explore Feed Use Case	Feed Layout Design	Display trending posts dynamically	Feed Module Developed	TC-10
ID-11	Receive real-time notifications	Notification Use Case	Notification System Design	Push notifications for events	Notification Service Developed	TC-11
ID-12	View notifications on a dedicated page	Notification Page Use Case	UI Notification Page	Display all notifications	Notification Module Developed	TC-12
ID-13	Delete notifications or mark as read	Notification Management Use Case	Interaction Design	Mark or remove notifications	Notification Control Module Developed	TC-13
ID-14	Upload images with multi-format support	Media Upload Use Case	Upload UI Design	Handle image uploads and formats	Media Service Developed	TC-14
ID-15	Protect media from unauthorized access	Media Security Use Case	Access Control Design	Secure media access system	Security Module Developed	TC-15
ID-16	Log system activities and errors	Logging Use Case	Logging System Design	Record system actions and errors	Logging Module Developed	TC-16
ID-17	Execute scheduled tasks (delete expired stories)	Scheduler Use Case	Scheduler Design	Auto-delete stories after 24h	Scheduler Service Developed	TC-17
ID-18	Report inappropriate or violating posts	Moderation Use Case	Report UI Design	Report system for posts	Moderation Module Developed	TC-18
ID-19	Suggest users/groups based on interests	Recommendation Use Case	Recommendation Engine Design	Suggest users algorithmically	Recommendation Module Developed	TC-19
ID-20	Discover trending content in real time	Trending System Use Case	Feed Algorithm Design	Real-time trending detection	Feed Engine Developed	TC-20

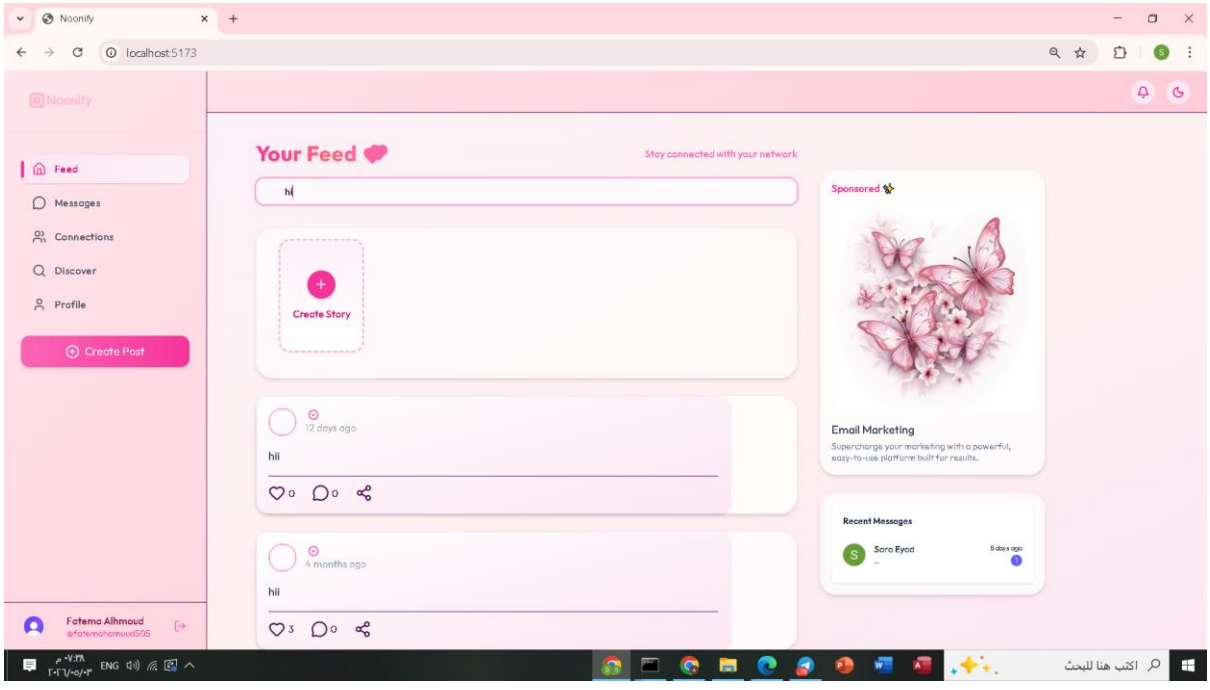
Chapter Four: Project Implementation

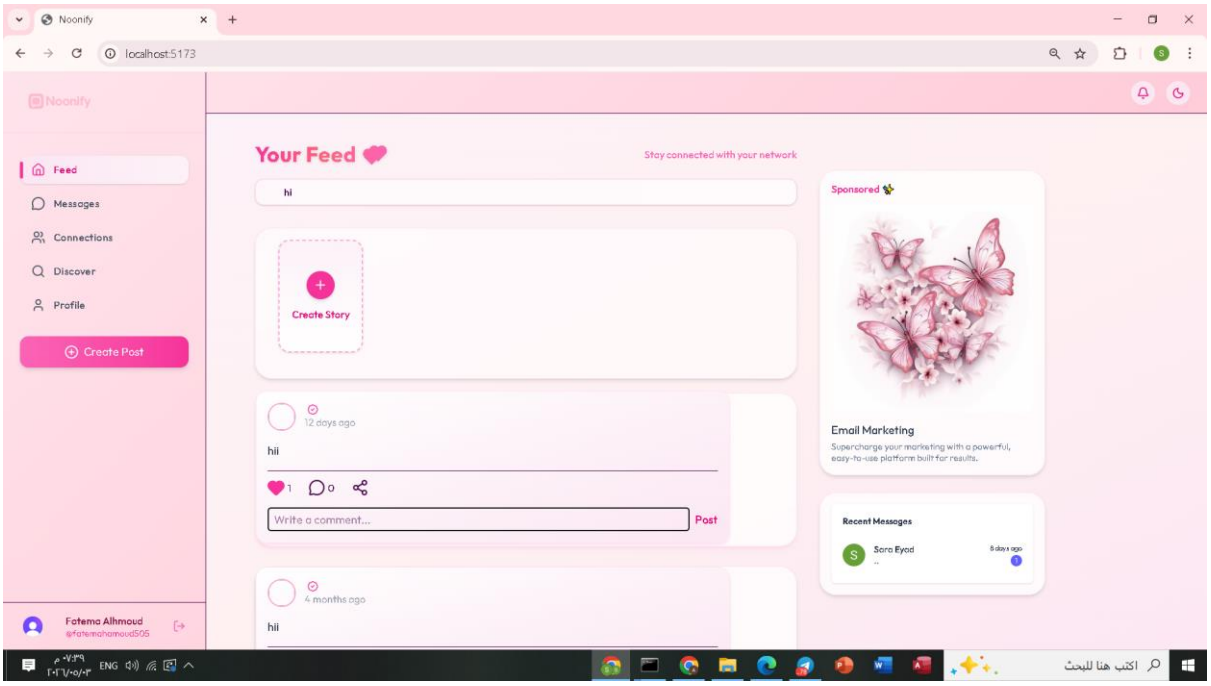
User Interfaces

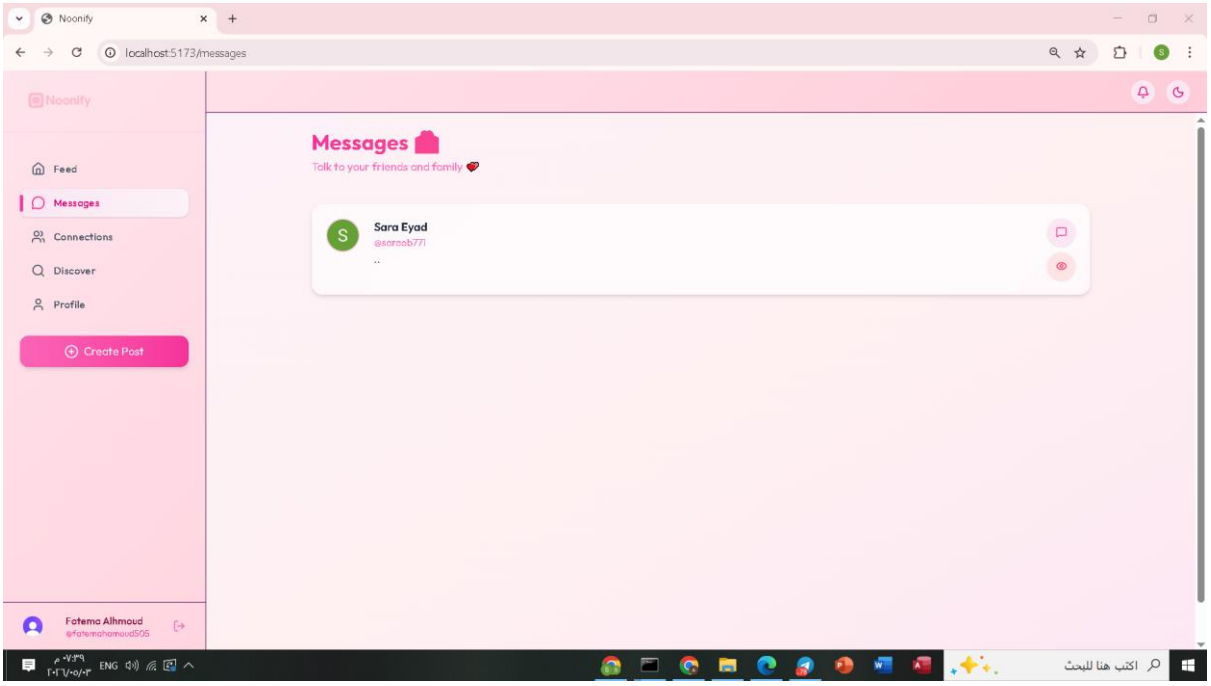


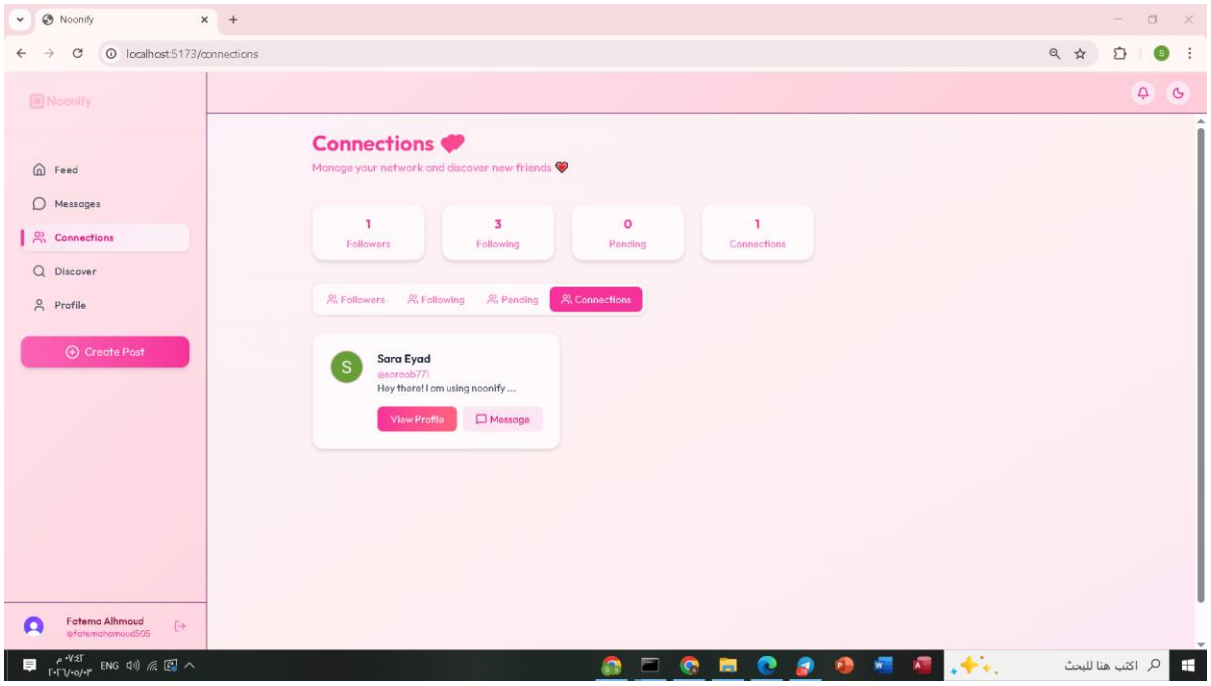


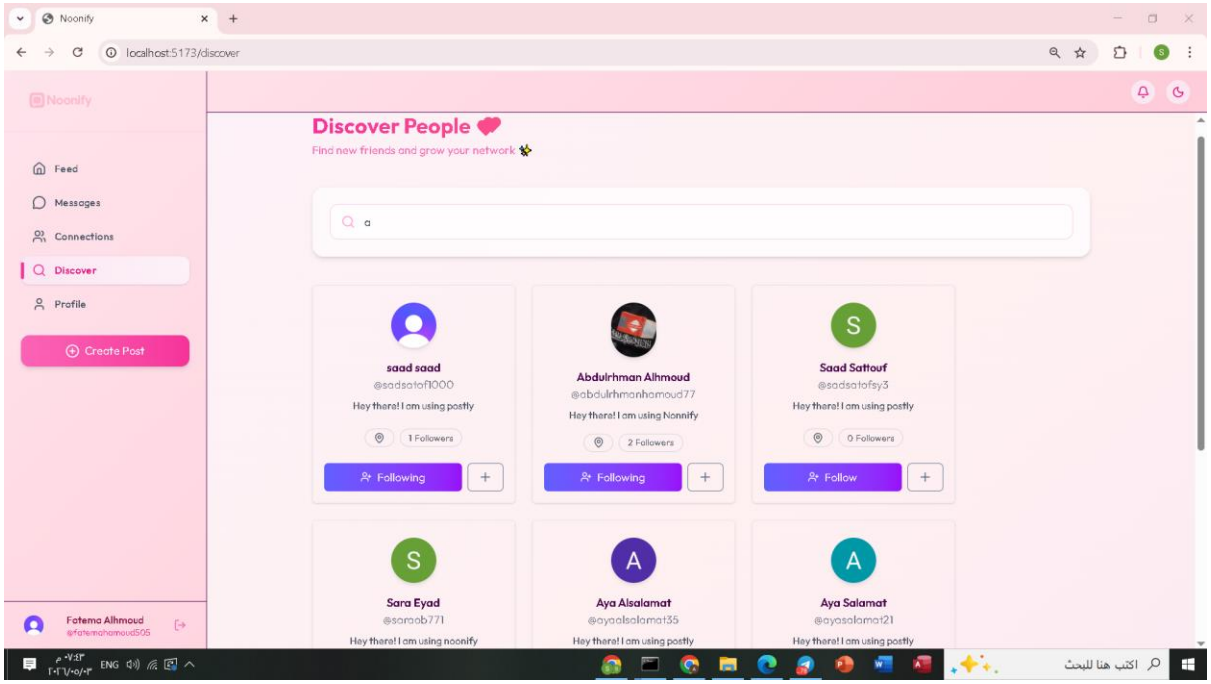

Noonify
localhost:5173
Noonify
Feed
Messages
Connections
Discover
Profile
Create Post
Your Feed
Stay connected with your network
Search posts, people...
Create Story
5 months ago
Noonify is Wowww!!!!
3
4
7 months ago
Hi
3
2
Sponsored
Email Marketing
Supercharge your marketing with a powerful, easy-to-use platform built for results.
Recent Messages
Sara Eyad
5 days ago
Fatema Alhmoud
@fatemahamoud505
ENG
اكتب هنا للبحث

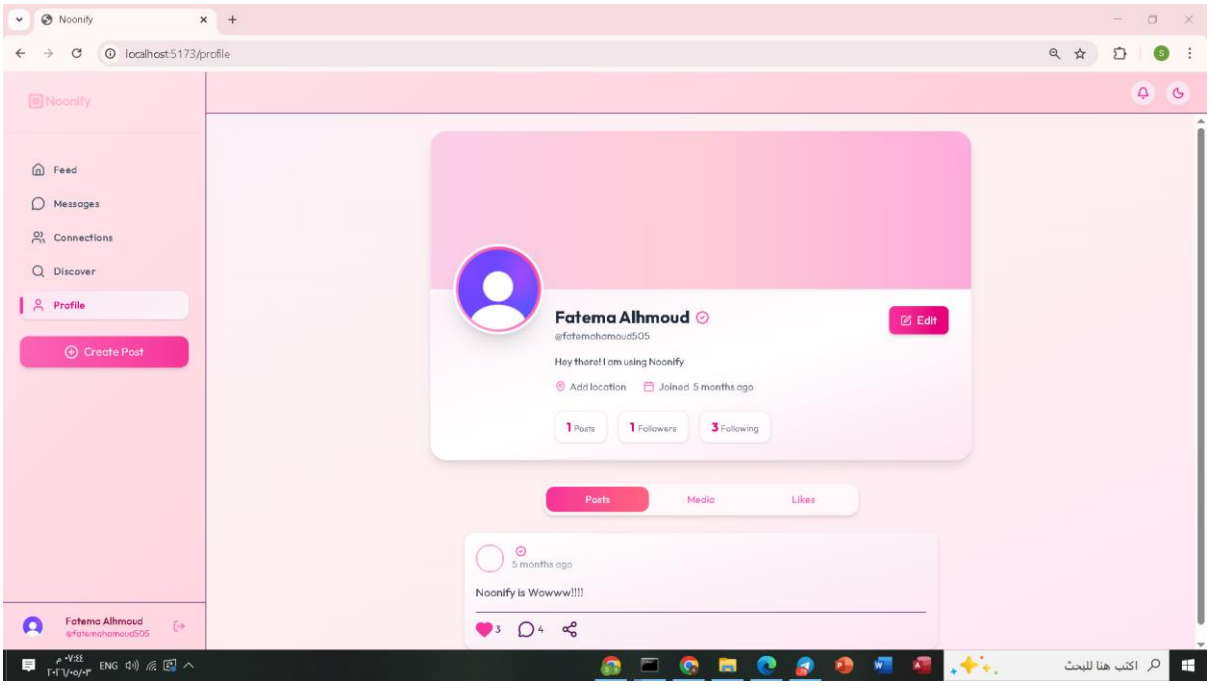

Noonify
localhost:5173
Noonify
Feed
Messages
Connections
Discover
Profile
Create Post
Your Feed
Stay connected with your network
hi
Create Story
12 days ago
hii
0
0
4 months ago
hii
3
0
Sponsored
Email Marketing
Supercharge your marketing with a powerful, easy-to-use platform built for results.
Recent Messages
Sara Eyad
5 days ago
Fatema Alhmoud
@fatemahamoud505
ENG
اكتب هنا للبحث

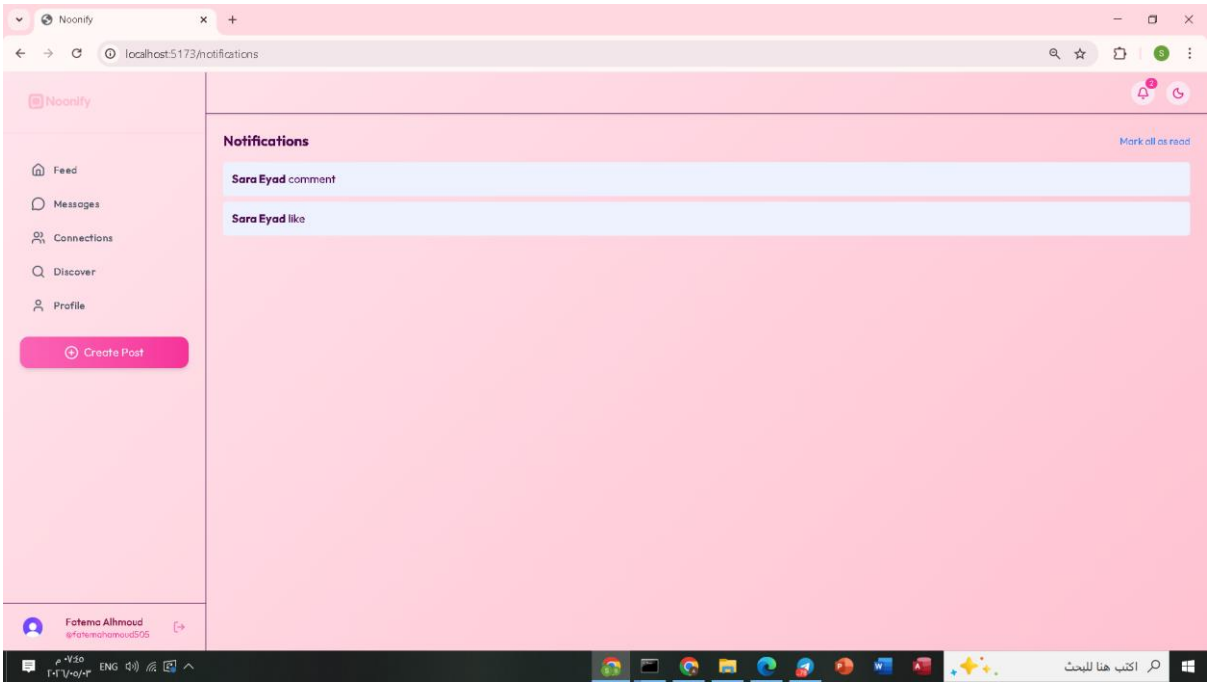

Noonify
localhost:5173
Noonify
Feed
Messages
Connections
Discover
Profile
Create Post
Your Feed
Stay connected with your network
hi
Create Story
12 days ago
hii
1
0
Write a comment...
Post
4 months ago
hii
Sponsored
Email Marketing
Supercharge your marketing with a powerful, easy-to-use platform built for results.
Recent Messages
Sara Eyad
Fatema Alhmoud
@fatemahamoud505
اكتب هنا للبحث

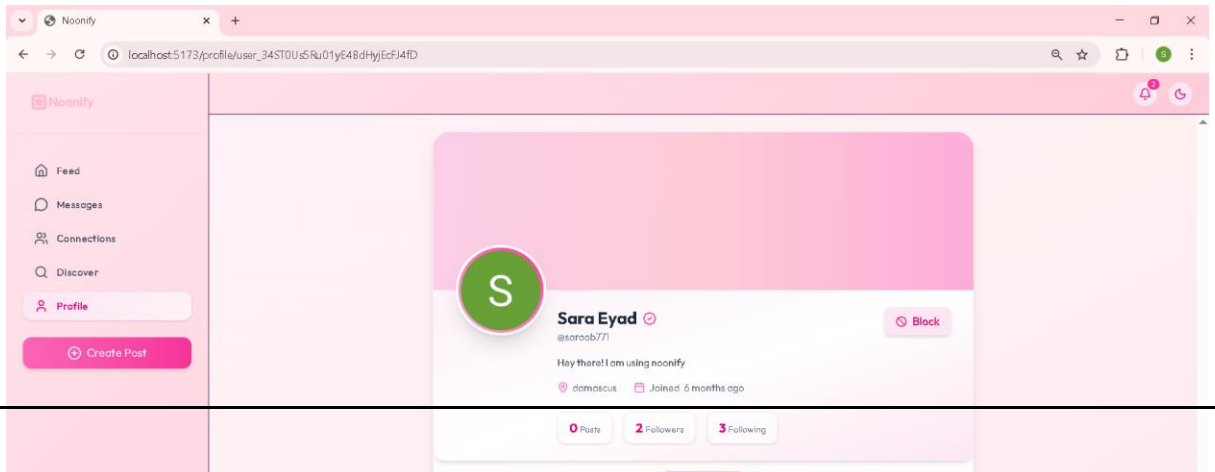

Noonify
localhost:5173/messages
Noonify
Feed
Messages
Connections
Discover
Profile
Create Post
Messages
Talk to your friends and family
Sara Eyad
@saraeb771
Fatema Alhmoud
@fatemahamoud505
اكتب هنا للبحث

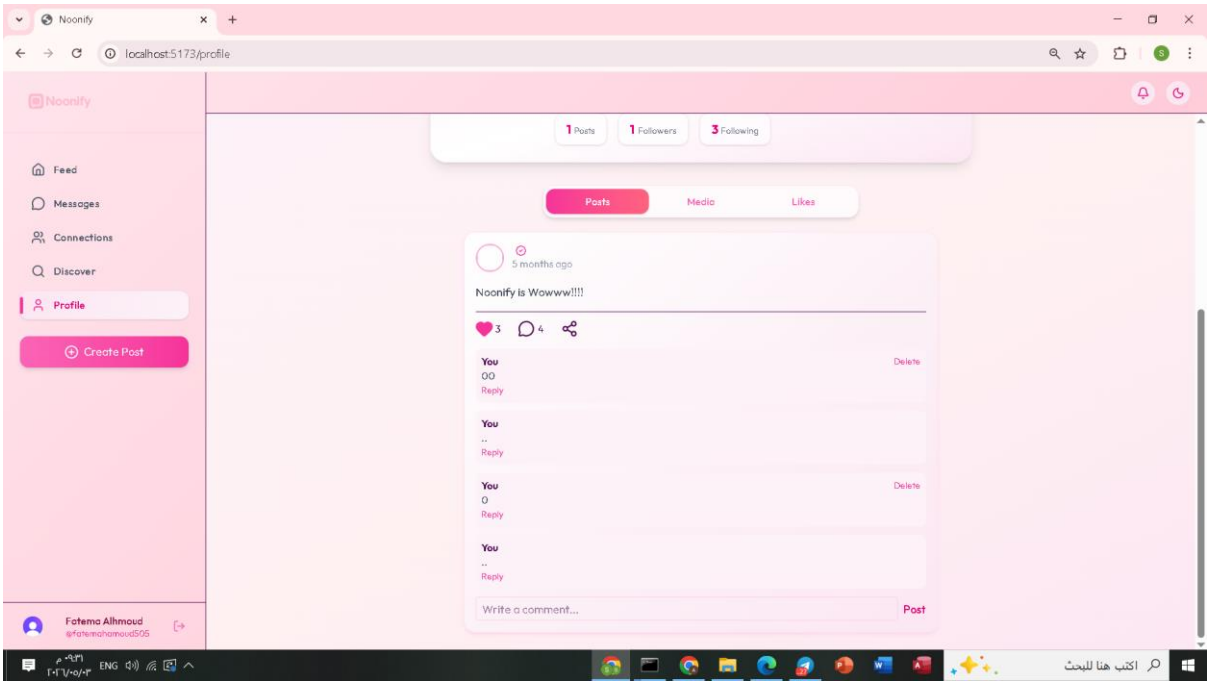

Noonify
localhost:5173/connections
Noonify
Feed
Messages
Connections
Discover
Profile
Create Post
Connections
Manage your network and discover new friends
1
Followers
3
Following
0
Pending
1
Connections
Followers
Following
Pending
Connections
Sara Eyad
@saraab771
Hey there! I am using noonify ...
View Profile
Message
Fatema Alhmoud
@fatemahamoud505
اكتب هنا للبحث

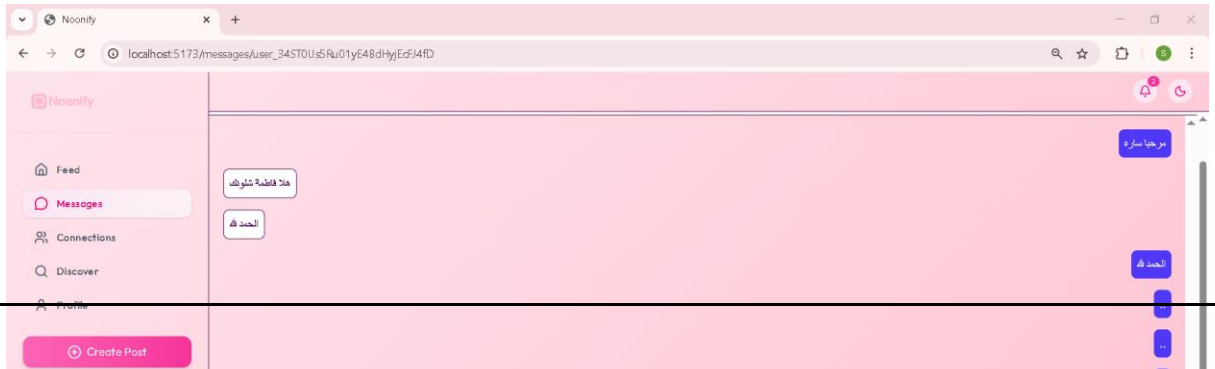

Noonify
localhost:5173/discover
Noonify
Feed
Messages
Connections
Discover
Profile
Create Post
Discover People
Find new friends and grow your network
a
saad saad
@sadsatof1000
Hey there! I am using postly
1 Followers
Following
Abdulrhman Alhmoud
@abdulrhmanhamoud77
Hey there! I am using Nonnify
2 Followers
Following
Saad Sattouf
@sadsatofsy3
Hey there! I am using postly
0 Followers
Follow
Sara Eyad
@saraab771
Hey there! I am using noonify
Aya Alsalamat
@ayaalsalamat35
Hey there! I am using postly
Aya Salamat
@ayasalamat21
Hey there! I am using postly
Fatema Alhmoud
@fatemahamoud505
اكتب هنا للبحث

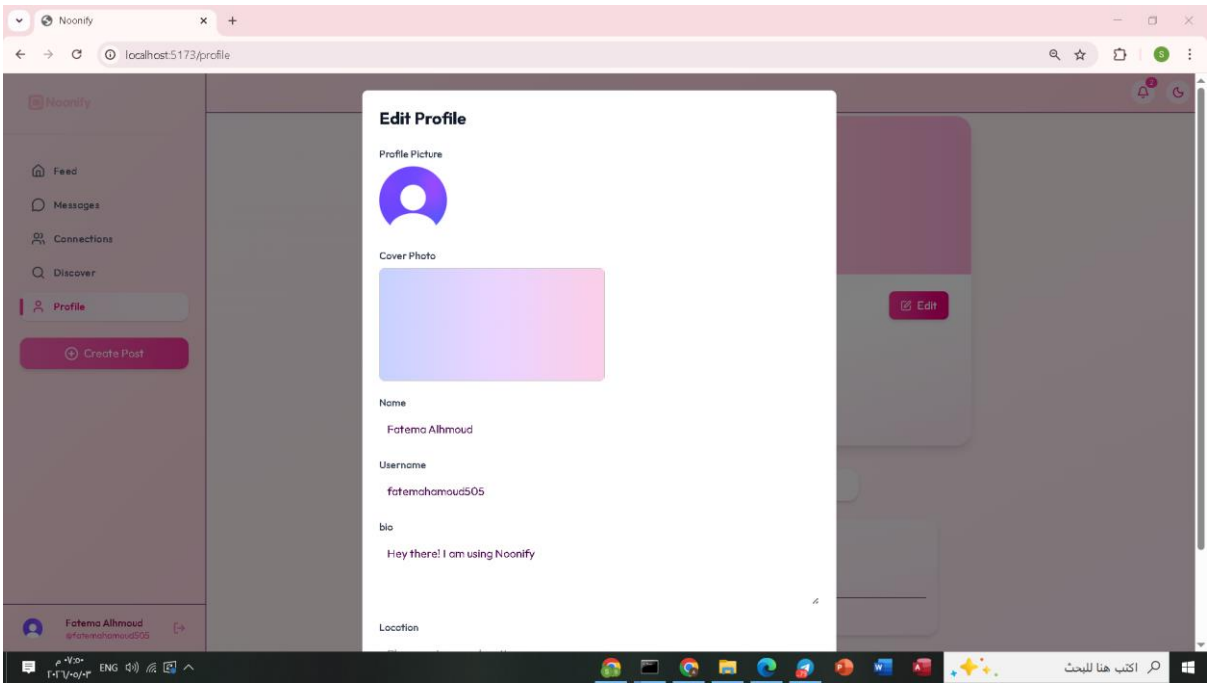

Noonify
localhost:5173/profile
Feed
Messages
Connections
Discover
Profile
Create Post
Fatema Alhmoud
@fatemahamoud505
Edit
Hey there! I am using Noonify
Add location
Joined 5 months ago
1 Posts
1 Followers
3 Following
Posts
Media
Likes
5 months ago
Noonify is Wowww!!!!
3
4
Fatema Alhmoud
@fatemahamoud505
ENG
اكتب هنا للبحث


Noonify
localhost:5173/notifications
Feed
Messages
Connections
Discover
Profile
Create Post
Notifications
Mark all as read
Sara Eyad comment
Sara Eyad like
Fatema Alhmoud
@fatemahamoud505
ENG
اكتب هنا للبحث


Noonify
localhost:5173/profile/user_34ST0Us5Ru01yE4BdHyjEcFJ4fD
Feed
Messages
Connections
Discover
Profile
Create Post
S
Sara Eyad
@saraab771
Block
Hey there! I am using noonify
damascus
Joined 6 months ago
0 Posts
2 Followers
3 Following


Noonify
localhost:5173/profile
Noonify
Feed
Messages
Connections
Discover
Profile
Create Post
1 Posts
1 Followers
3 Following
Posts
Media
Likes
5 months ago
Noonify is Wowww!!!!
3
4
You
00
Reply
Delete
You
..
Reply
You
0
Reply
Delete
You
..
Reply
Write a comment...
Post
Fatema Alhmoud
@fatemahamoud505
ENG
اكتب هنا للبحث


Noonify
localhost:5173/messages/user_34ST0Us5Ru01yE4BdHyjEcFJ4fD
Noonify
Feed
Messages
Connections
Discover
Profile
Create Post
مرحبا سارة
هلا فاطمة شلونك
الحمد لله
الحمد لله


Noonify
localhost:5173/profile
Noonify
Feed
Messages
Connections
Discover
Profile
Create Post
Fatema Alhmoud
Edit Profile
Profile Picture
Cover Photo
Name
Fatema Alhmoud
Username
fatemahamoud505
bio
Hey there! I am using Noonify
Location
Edit
اكتب هنا للبحث

Conclusion

In conclusion, this project successfully designed and developed a comprehensive social media platform named **NOONIFY**, aiming to replicate the core features of modern social networking systems while addressing both academic and practical aspects. The project was built using the MERN Stack architecture, which integrates an interactive front-end developed with React.js, a back-end powered by Node.js and Express.js, and a MongoDB database. This combination enabled the creation of a flexible, well-structured, and scalable system.

The project achieved its defined objectives by allowing users to create personal accounts, manage their profiles, share textual and multimedia content, interact with posts, use the stories feature, and communicate through private messaging. Additionally, key architectural concepts such as Client–Server Architecture and MVC Architecture were implemented, contributing to better code organization, improved maintainability, and easier future feature development.

Furthermore, this project enhanced practical skills in modern web application development, working within the Scrum methodology, and requirements analysis. It serves as a practical model that bridges the gap between theoretical knowledge and real-world application in the fields of software engineering and information systems.

Future Work

Despite achieving its primary goals, there are several potential improvements and extensions that can be implemented in the future to enhance the platform's efficiency and expand its capabilities, including:

- **Enhancing Security**
- Implement advanced security techniques such as Two-Factor Authentication and improved encryption policies to protect user data.
- **Improving Performance and Scalability**

Support caching techniques and utilize advanced cloud services to enable the platform to handle a larger number of users.

- **Integrating Artificial Intelligence**

Incorporate recommendation algorithms to suggest friends and relevant content based on user interests.

- **Developing Mobile Applications**

Build dedicated applications for Android and iOS to provide a smoother user experience.

- **Expanding Social Interaction Features**

Support live streaming, push notifications, and enhance the commenting and interaction systems.

- **Improving User Experience (UX/UI)**

Conduct real user studies to improve interface design and make it more intuitive and user-friendly.

References

1. Books and Articles

- Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.
- Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson Education.

2. Online Resources

- MongoDB Inc. (n.d.). *MongoDB Documentation*. Retrieved from https://www.mongodb.com/docs/
- React. (n.d.). *React Official Documentation*. Retrieved from https://react.dev/
- Node.js Foundation. (n.d.). *Node.js Documentation*. Retrieved from https://nodejs.org/en/docs
- Express.js. (n.d.). *Express Framework Documentation*. Retrieved from https://expressjs.com/

3. Tools and Software Documentation

- Vite. (n.d.). *Vite – Next Generation Frontend Tooling*. Retrieved from https://vitejs.dev/
- Mongoose. (n.d.). *Mongoose ODM Documentation*. Retrieved from https://mongoosejs.com/docs/
- NodeMailer. (n.d.). *NodeMailer Documentation*. Retrieved from https://nodemailer.com/about/

4. Additional References

- Scrum.org. (n.d.). *Scrum Guide*. Retrieved from https://www.scrumguides.org/
- Mozilla Developer Network (MDN). (n.d.). *Web Technologies Documentation*. Retrieved from https://developer.mozilla.org/
- ImageKit. (n.d.). *ImageKit Media Optimization Documentation*. Retrieved from https://imagekit.io/docs

Appendix: Junior Project Report

The appendix includes supplementary materials related to the Junior project, such as system screenshots, interface designs, diagrams (Use Case, ERD, Class Diagram), test cases, and additional implementation details that support the main content of the report.

i
ii